



INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO PIAUÍ

Campus Piripiri

Tecnólogo em Análise e Desenvolvimento de Sistemas

Francisco Igor Silva Santos

Sistema para Emissão, Gestão e Autenticação de Certificados

Piripiri – PI

2026

Francisco Igor Silva Santos

2024116TADS0030

Sistema para Emissão, Gestão e Autenticação de Certificados

Relatório Técnico de Software apresentado ao curso de Tecnólogo em Análise e Desenvolvimento de Sistemas, como requisito parcial para a conclusão do Trabalho de Conclusão de Curso (TCC).

Orientador(a): Prof. Mayllon Veras da Silva

Resumo

Texto resumido do projeto, objetivos, tecnologias e resultados. Palavras-chave: software, desenvolvimento, tecnologia.

Abstract

Short text in English summarizing the work. Keywords: software, development, technology.

Sumário

1.0 Introdução	6
1.1 Objetivos	7
1.1.1 Geral	7
1.1.2 Específicos	7
2.0 Tecnologias Envolvidas	8
2.1 Frontend	8
2.1.1 React 19	8
2.1.2 TypeScript	9
2.1.3 Tailwind CSS	9
2.1.4 shadcn/ui	10
2.1.5 TanStack Router	10
2.1.6 React Hook Form	11
2.1.7 Sonner	11
2.1.8 Recharts	11
2.1.9 date-fns	12
2.1.10 Vite	12
2.1.11 Arquitetura e Padrões	13
2.2 Integração com a API REST	13
2.2.1 Zod	14
2.3 Backend	14
2.3.1 Next.js 16	14
2.3.2 Supabase e PostgreSQL	15
2.3.3 PDFKit	16
2.3.4 Zod	17
2.3.5 Arquitetura e Padrões	17
2.4 Infraestrutura e Ferramentas	17
2.4.1 Render	17
2.4.2 Jest	18
2.4.3 ESLint e Prettier	18
2.4.4 Git e GitHub	19

2.4.5	Visual Studio Code	19
2.5	Síntese das Tecnologias	19
3.0	Modelagem do Projeto	21
3.1	Levantamento de Requisitos	21
3.1.1	Módulo de Autenticação e Controle de Acesso	21
3.1.2	Módulo de Templates de Certificados	22
3.1.3	Módulo de Entrada de Dados	22
3.1.4	Módulo de Geração de Certificados	22
3.1.5	Módulo de Envio e Comunicação	23
3.1.6	Módulo de Validação Pública e Antifraude	23
3.1.7	Módulo de Assinaturas Digitais	23
3.1.8	Módulo de Dashboard e Gestão	23
3.1.9	Módulo de Auditoria e Segurança Operacional	24
3.1.10	Módulo de Gestão de Cursos/Eventos	24
3.1.11	Módulo de Participantes	24
3.1.12	Módulo Multiempresa / Multi-tenant	25
3.1.13	Módulo de Configurações do Sistema	25
3.1.14	Módulo de Armazenamento e Arquivos	25
3.1.15	Módulo de Notificações e Alertas	25
3.1.16	Módulo de Monitoramento e Logs	25
3.1.17	Módulo de Suporte, Operação e Manutenção	26
3.1.18	Segurança	26
3.1.19	Desempenho e Escalabilidade	27
3.1.20	Disponibilidade e Confiabilidade	27
3.1.21	Usabilidade e Acessibilidade	28
3.1.22	Compatibilidade e Integração	28
3.1.23	Armazenamento e Retenção	28
3.1.24	Conformidade Legal e LGPD	28
3.1.25	Arquitetura e Tecnologia	29
3.1.26	Manutenção e Monitoramento	30
3.1.27	Internacionalização	30
3.2	Regras de Negócio	30

3.3	Critérios de Aceitação	31
3.4	Matriz de Rastreabilidade	35
3.5	Diagramas de casos de uso	36
3.6	Diagramas de classe	49
3.7	Arquitetura do Sistema	52
3.8	Diagrama Entidade-Relacionamento	56
3.9	Interfaces	60
4.0	Software	60
4.1	Implantação	60
4.2	Testes	61
4.2.1	Plano de Testes	62
4.2.2	Frontend (Certificate Hub)	66
4.2.3	Backend (Certificate Server)	67
4.2.4	Testes End-to-End	68
4.2.5	Limitações	69
4.2.6	Resumo Geral	70
5.0	Considerações Finais	70
	Referências	73
6.0	Anexos	74
6.1	Declaração de Entrega do Código-Fonte	74
6.2	Declaração de Submissão ao NIT	74

1.0 Introdução

A emissão de certificados é uma atividade essencial para empresas de treinamentos, consultorias e instituições educacionais que promovem cursos, capacitações e eventos. Apesar da relevância desse processo, grande parte das organizações ainda depende de procedimentos manuais ou soluções improvisadas, utilizando ferramentas como Google Planilhas, Google Docs, PowerPoint e complementos externos.

Essas práticas resultam em retrabalho, inconsistências, falta de padronização e dificuldade de controle operacional. Além disso, ferramentas como AutoCrat e documentos espalhados entre diferentes plataformas não foram projetados para uso profissional em escala, o que gera limitação, fragilidade e risco de inconsistências.

A ausência de um sistema integrado compromete a rastreabilidade e a credibilidade dos certificados, uma vez que não há mecanismos oficiais de validação pública, histórico centralizado, indicadores gerenciais ou emissão em lote eficiente. Estudos recentes mostram que sistemas baseados em QR Code têm se destacado como alternativa eficaz para verificação de credenciais acadêmicas [1], combinando criptografia e códigos bidimensionais para garantir a autenticidade dos documentos [2]. Esse cenário se agrava com o crescimento de cursos online e da necessidade de certificação confiável.

Diante disso, justifica-se o desenvolvimento de uma solução centralizada e automatizada, capaz de reduzir erros humanos, otimizar o tempo operacional e garantir segurança, conformidade e profissionalismo. Recursos como códigos únicos, validação via QR Code e assinaturas digitais ampliam a confiabilidade dos certificados e fortalecem a imagem institucional das organizações emissoras [3]. Soluções semelhantes já foram implementadas com sucesso utilizando criptografia AES e QR Code para verificação de certificados acadêmicos [4].

O presente trabalho consiste no desenvolvimento de um Sistema Web — produto do tipo SaaS (Software as a Service) com arquitetura multi-tenant — destinado à emissão, gestão e autenticação de certificados. O sistema foi projetado com base nas necessidades reais de uma empresa parceira do ramo de treinamentos corporativos, que atualmente enfrenta as limitações operacionais descritas. Dessa forma, o escopo abrange desde a automatização da criação e personalização de certificados até a disponibilização de um portal público de validação, visando substituir processos manuais por uma plataforma profissional, segura e escalável.

O desenvolvimento seguiu a abordagem de Produto Mínimo Viável (MVP), priorizando as funcionalidades essenciais para a operação do sistema — emissão de certificados, templa-

tes configuráveis, dashboard e validação pública — enquanto funcionalidades complementares, como importação em lote e envio automático por e-mail, foram postergadas para versões futuras. O modelo SaaS adotado permite que o sistema seja oferecido como serviço centralizado, simplificando a manutenção, a escalabilidade e a atualização contínua da plataforma.

Este relatório técnico está organizado da seguinte forma: a Seção 2 apresenta as tecnologias utilizadas no desenvolvimento; a Seção 3 descreve a modelagem do sistema, incluindo levantamento de requisitos, diagramas de casos de uso, classes, arquitetura e entidade-relacionamento; a Seção 4 detalha a implementação do software; e a Seção 5 traz as considerações finais.

1.1 Objetivos

1.1.1 Geral

Projetar e implementar um sistema web automatizado para emissão, gestão, envio e validação de certificados, destinado a empresas de treinamento corporativo, instituições educacionais e demais organizações que necessitem certificar participantes em cursos, capacitações e eventos, a fim de eliminar a dependência de processos manuais e soluções improvisadas, garantindo padronização dos documentos, rastreabilidade do histórico de emissões, autenticação confiável via código único e QR Code, emissão em lote eficiente e centralização dos dados em uma única plataforma.

1.1.2 Específicos

- implementar um módulo de templates configuráveis para a criação de certificados;
- desenvolver a funcionalidade de importação de dados a partir de arquivos CSV e Excel (.xlsx) com processamento em fila para emissão em lote;
- implementar um módulo de notificação por e-mail com suporte a variáveis dinâmicas e reenvio automático em caso de falha;
- desenvolver um mecanismo de autenticação pública via código único e QR Code para garantir a integridade dos certificados e prevenir fraudes;
- desenvolver um módulo de dashboard com indicadores gerenciais, filtros por período, curso e status;

- planejar e executar testes unitários e de integração para validação dos módulos implementados;
- configurar o ambiente de implantação do sistema em infraestrutura cloud com deploys automáticos a partir do repositório Git.

2.0 Tecnologias Envolvidas

Este capítulo apresenta as tecnologias empregadas no desenvolvimento do sistema, organizadas em frontend, backend e infraestrutura, com uma discussão sobre o papel de cada uma, os motivos de sua escolha e as alternativas consideradas durante o processo de decisão. As seções a seguir detalham as tecnologias de cada camada.

2.1 Frontend

A camada de apresentação foi construída com tecnologias modernas do ecossistema JavaScript, priorizando tipagem estática, experiência de desenvolvimento produtiva e performance em tempo de execução.

2.1.1 React 19

React é uma biblioteca JavaScript para construção de interfaces de usuário baseada no conceito de componentes reutilizáveis e estado unidirecional [5]. A versão 19 introduz melhorias significativas no modelo de concorrência, como o novo hook `use()`, que permite leitura assíncrona de recursos diretamente no corpo do componente, e as *Actions* para gerenciamento nativo de formulários, reduzindo a necessidade de bibliotecas externas para controle de estado de formulários.

A escolha do React 19 baseou-se em dois fatores principais: (1) maturidade do ecossistema, com ampla disponibilidade de bibliotecas e documentação; e (2) modelo de componentes que favorece a modularidade e a testabilidade do código. Alternativas como Vue.js e Angular foram descartadas respectivamente pelo ecossistema menos consolidado para o tipo de aplicação e pela curva de aprendizado mais steep.

2.1.2 TypeScript

TypeScript é um superconjunto tipado de JavaScript que compila para JavaScript puro. Seu sistema de tipos estrutural permite detectar erros comuns em tempo de compilação, como acesso a propriedades inexistentes, passagem de argumentos com tipos incompatíveis e retorno incorreto de funções [6].

A adoção do TypeScript em vez de JavaScript puro deveu-se à necessidade de contratos explícitos entre módulos e à segurança proporcionada pela verificação estática de tipos: erros como acesso a propriedades inexistentes ou argumentos com tipos incorretos são detectados em tempo de compilação, e não em runtime. Alternativas como Flow (Facebook) foram descartadas por seu ecossistema significativamente menor — o Flow não é suportado nativamente por ferramentas como ESLint, Prettier e o próprio Vite — e pela comunidade majoritariamente concentrada no TypeScript, que resulta em melhor documentação, mais tipos publicados (@types/*) e suporte de primeira classe em IDEs.

No projeto, o TypeScript foi utilizado em toda a camada frontend. O recurso de *generics* possibilitou a criação de hooks e utilitários reutilizáveis com segurança de tipos, enquanto *union types* e *discriminated unions* foram empregados para modelar estados de carregamento, erro e sucesso das requisições assíncronas.

2.1.3 Tailwind CSS

Tailwind CSS é um framework de estilização baseado no paradigma *utility-first*, fornecendo classes atômicas de baixo nível (como `flex`, `p-4`, `text-lg`) que são compostas diretamente no markup HTML/JSX [7]. Diferentemente de abordagens tradicionais como Bootstrap (baseada em componentes pré-construídos), o Tailwind oferece maior flexibilidade de design sem a necessidade de sobrescrever estilos padrão.

Na versão 4, o Tailwind adota uma abordagem *CSS-first*, eliminando a necessidade do arquivo `tailwind.config.js`. A configuração é feita diretamente via CSS com o diretório `@theme`, e o *tree-shaking* é automático: o plugin `@tailwindcss/vite` detecta quais classes são usadas no código-fonte e gera apenas o CSS necessário, resultando em bundles tipicamente inferiores a 10 KB. A consistência visual foi garantida pela definição de um tema personalizado com cores, espaçamentos e tipografia alinhados à identidade visual do sistema.

2.1.4 shadcn/ui

shadcn/ui é uma coleção de componentes reutilizáveis construídos sobre *Radix UI* (primitivas acessíveis e sem estilo) e estilizados com Tailwind CSS [8]. Diferentemente de bibliotecas tradicionais como Material UI ou Chakra UI, o shadcn/ui não é publicado como um pacote npm instalável — os componentes são copiados diretamente para o código-fonte do projeto, concedendo controle total sobre a implementação e personalização.

No sistema, foram utilizados 46 componentes do shadcn/ui: `Button`, `Card`, `Dialog`, `Form`, `Table`, `Sidebar`, `Chart` e `Toast`. O utilitário `class-variance-authority` gerencia variantes de componentes. As classes CSS são resolvidas com `tailwind-merge` e `clsx`. O `Button` oferece as variantes `default`, `destructive`, `outline`, `secondary`, `ghost` e `link`, definidas via `cva()`.

2.1.5 TanStack Router

O TanStack Router é uma biblioteca de roteamento para React que adota uma abordagem declarativa e baseada em arquivos para definição de rotas [9]. Seu diferencial em relação ao React Router DOM tradicional está no suporte a *search params* tipados, *loaders* e *actions* que integram o carregamento de dados ao ciclo de vida da navegação.

No sistema, as rotas foram organizadas em uma árvore aninhada que reflete a hierarquia de navegação do usuário: rotas públicas (login, validação de certificados), rotas protegidas (dashboard, gestão de eventos) e rotas administrativas (configurações, relatórios). O recurso de *beforeLoad* foi utilizado para verificar a autenticação antes de renderizar qualquer rota protegida, redirecionando usuários não autenticados para a tela de login sem necessidade de estado global.

O React Router DOM, embora mais difundido, foi preterido por não oferecer tipagem nativa para *search params* e *path params* — o desenvolvedor precisa declarar tipos manualmente ou usar bibliotecas auxiliares como `zod-router`. O TanStack Router infere os tipos automaticamente a partir da definição da rota, eliminando uma fonte comum de erros em tempo de execução. Além disso, seu modelo de *loaders* e *actions* elimina a necessidade de `useEffect` para buscar dados durante a navegação, substituindo-o por funções assíncronas executadas antes da renderização da rota.

2.1.6 React Hook Form

React Hook Form é uma biblioteca para gerenciamento de formulários em React que minimiza o número de re-renderizações ao utilizar refs em vez de estado controlado [10]. Diferentemente do Formik, que mantém o estado do formulário no React state e dispara uma re-renderização a cada tecla digitada, o React Hook Form registra os campos via refs e só re-renderiza componentes específicos quando necessário — o que resulta em melhor desempenho em formulários com muitos campos. Sua integração com o Zod via `@hookform/resolvers` permite que os esquemas de validação definidos com Zod sejam reutilizados como regras de formulário, garantindo que as mesmas validações aplicadas no backend sejam executadas no frontend antes do envio.

No sistema, o padrão adotado combina `useForm` com `zodResolver` para cada formulário (emissão de certificado, cadastro de template, etc.). As mensagens de erro retornadas pelo Zod são mapeadas automaticamente para os campos correspondentes e exibidas via componentes `FormMessage` do `shadcn/ui`. O modo `onBlur` dispara a validação no momento em que o usuário sai do campo, oferecendo feedback imediato sem a latência da validação a cada tecla.

2.1.7 Sonner

Sonner é uma biblioteca leve de notificações *toast* para React [11]. Diferentemente de alternativas como `react-hot-toast` e `notistack`, o Sonner prioriza simplicidade (exporta apenas `toast` e `Toaster`) e acessibilidade com suporte a leitores de tela.

No sistema, o Sonner é utilizado para exibir feedback ao usuário após operações assíncronas: sucesso na emissão de certificados, erros de validação, confirmação de exclusão e falhas de rede. As notificações são posicionadas no canto superior direito e configuradas para desaparecer automaticamente após 4 segundos, exceto erros que requerem ação manual.

2.1.8 Recharts

Recharts é uma biblioteca de gráficos para React construída sobre os componentes SVG do D3, seguindo uma abordagem declarativa e composável [12]. Cada tipo de gráfico é um componente React independente (`BarChart`, `PieChart`, `LineChart`), facilitando a personalização sem conhecimento aprofundado de SVG.

A escolha do Recharts em vez do `react-chartjs-2` (wrapper para `Chart.js`) deu-se pela integração mais natural com o ecossistema React: enquanto o `Chart.js` manipula o canvas diretamente e

exige imperativamente chamar `update()` para refletir alterações nos dados, o Recharts utiliza SVG declarativo e reage automaticamente a mudanças de props — o que simplifica a manutenção de gráficos dinâmicos que se atualizam conforme o usuário filtra os dados no dashboard. O D3 puro foi descartado por sua API de baixo nível, que demandaria mais código boilerplate para alcançar o mesmo resultado.

No sistema, o Recharts é utilizado no dashboard para exibir métricas como total de certificados emitidos por mês (gráfico de barras) e distribuição por tipo de template (gráfico de pizza). Os componentes `ResponsiveContainer`, `Tooltip` e `Legend` garantem que os gráficos se adaptem ao tamanho da tela e exibam informações contextuais ao passar o mouse.

2.1.9 date-fns

`date-fns` é uma biblioteca de utilitários para manipulação de datas em JavaScript, caracterizada por sua abordagem modular e funcional [13]. Cada função é importada individualmente (`format`, `parse`, `differenceInDays`, `isAfter`), evitando o aumento desnecessário do bundle com código não utilizado.

Alternativas como `Moment.js` foram descartadas porque o `Moment.js` é considerado obsoleto pela própria equipe de mantenedores (modo de manutenção desde 2020), possui API mutável que pode causar efeitos colaterais inesperados e não suporta *tree-shaking* — importar uma única função inclui a biblioteca inteira no bundle final. O `dayjs`, embora mais leve que o `Moment.js`, foi preterido por sua API menos expressiva para operações como *locale* customizado e formatação avançada. O `date-fns`, por sua vez, adota funções puras (sem mutação), é completamente modular e oferece suporte a 80+ locales, incluindo português brasileiro.

No sistema, o `date-fns` é empregado para formatar datas de validade de certificados nos componentes de listagem, calcular diferenças entre datas para alertas de expiração e padronizar a exibição de datas no formato local (`dd/MM/yyyy`) via função `format`. A função `isAfter` é utilizada no modelo `certificate.ts` para determinar se um certificado está expirado.

2.1.10 Vite

Vite é uma ferramenta de build e servidor de desenvolvimento que utiliza módulos ES nativos durante o desenvolvimento para oferecer *Hot Module Replacement* (HMR) instantâneo, eliminando a necessidade de empacotamento em tempo de desenvolvimento [14]. Em produção, utiliza Rollup para gerar bundles otimizados com *code splitting* automático e *lazy loading*.

No projeto, o Vite é a ferramenta de build ****exclusiva do frontend SPA**** — o backend Next.js utiliza seu próprio sistema de compilação (`next build`). Essa divisão reforça a independência entre as duas aplicações: alterações no frontend não exigem reconstrução do backend, e vice-versa. A substituição do Create React App (CRA) pelo Vite reduziu o tempo de inicialização do servidor de desenvolvimento de aproximadamente 30 segundos para menos de 2 segundos, e o HMR reflete alterações no código em milissegundos, melhorando significativamente a produtividade durante o desenvolvimento.

2.1.11 Arquitetura e Padrões

O frontend adota uma arquitetura em camadas com React, organizada em quatro níveis. A camada *View* reúne os componentes React que renderizam a interface e capturam eventos do usuário, divididos entre páginas (`views/`) e componentes reutilizáveis (`components/`), incluindo os do `shadcn/ui`. A camada *ViewModel* é composta por custom hooks (`useCertificatesViewModel`, `useDashboardViewModel`, etc.) que encapsulam o estado local com `useState`, efeitos colaterais com `useEffect`, dados derivados com `useMemo` e expõem ações de forma coesa — cada hook retorna um objeto com o estado e os manipuladores necessários para sua respectiva tela. A camada *Service* centraliza a comunicação externa: o módulo `api.ts` utiliza a `fetch` API nativa para consumir a API REST, e o módulo `envios.ts` gerencia uma fila local de envios de e-mail no `localStorage`, armazenando apenas dados operacionais (nome e e-mail do destinatário, *status* da entrega). Não há armazenamento de tokens de autenticação ou credenciais no `localStorage` — a autenticação será gerenciada pelo Supabase Auth, que utiliza mecanismo de sessão próprio e seguro. A camada *Model* contém as interfaces TypeScript que definem as entidades de domínio (`Certificate`, `Template`, `VerifyResult`, `EnvioEmail`) com tipos inferidos. O TanStack Router orquestra a navegação, mapeando URLs a componentes (`views/`) de forma declarativa. O Tailwind CSS gerencia a estilização em todas as camadas de apresentação. Essa separação permite testar a lógica de apresentação independentemente da interface, utilizando Jest para validar os *ViewModels* sem depender de um navegador.

2.2 Integração com a API REST

O frontend comunica-se com um backend externo por meio de uma API REST, cuja URL base é configurada via variável de ambiente `VITE_API_URL`. As requisições são centralizadas

em um módulo `api.ts` que utiliza a `fetch` API nativa do JavaScript, encapsulando o tratamento de erros e a serialização JSON. Os endpoints expostos incluem operações CRUD para templates e certificados, emissão de certificados, geração de PDF e verificação de autenticidade.

2.2.1 Zod

Zod é uma biblioteca de validação de esquemas com suporte primário a TypeScript, que permite definir esquemas de dados com inferência automática de tipos [15]. Diferentemente de alternativas como Joi ou Yup, o Zod foi projetado especificamente para integração com TypeScript, eliminando a necessidade de declarar tipos manualmente: o tipo é inferido diretamente do esquema de validação.

No sistema, os esquemas Zod são utilizados em conjunto com o React Hook Form para validar as entradas dos formulários antes do envio à API. Um mesmo esquema pode validar desde o formato de e-mail até a estrutura completa de um cadastro de certificado, com mensagens de erro localizadas exibidas nos componentes `FormMessage` do `shadcn/ui`. Essa abordagem garante consistência entre a validação no frontend e as regras aplicadas pelo backend.

2.3 Backend

O backend do sistema é uma API REST desenvolvida com Next.js e executada no Node.js 22 LTS [16], seguindo uma arquitetura em camadas que separa o código em três camadas: domínio (*entities* e *value objects* sem dependências externas), aplicação (*use cases* e *DTOs*) e infraestrutura (*repositories*, *services* e acesso a banco de dados). As dependências apontam para dentro: a camada de domínio define interfaces que a infraestrutura implementa, permitindo substituir implementações sem alterar a lógica de negócio.

2.3.1 Next.js 16

Next.js é um framework React que, em sua versão 16, oferece o *App Router* com *Route Handlers* para construção de APIs serverless [17]. Cada endpoint é definido como um arquivo em `src/app/api/` exportando funções assíncronas nomeadas (GET, POST, PUT, DELETE) — não há necessidade de um servidor Express ou configuração de rotas manual.

É importante destacar que, neste projeto, o Next.js é utilizado ***exclusivamente* como servidor de API (*backend-only*)*: não há páginas, layouts, *Server Components* ou qualquer funcionalidade de renderização frontend. O diretório `src/app/` contém apenas subdiretórios `ap`

i / * com *Route Handlers*; não existe um `page.tsx` ou `layout.tsx` no projeto. O frontend é uma SPA completamente separada (descrita na Seção 2.1), que consome a API via HTTP. Essa separação foi intencional: o frontend SPA poderia ser substituído por outro cliente (como um aplicativo mobile) sem qualquer alteração no backend, e o backend poderia ser reescrito em outra linguagem sem afetar o frontend. O fato de o Next.js ser tipicamente associado a aplicações full-stack não invalida seu uso como servidor de API puro — os *Route Handlers* operam de forma independente do sistema de páginas e podem substituir frameworks como Express ou Fastify com menos configuração.

No sistema, os *Route Handlers* atuam como a camada mais externa: recebem a requisição, validam os parâmetros com Zod, instanciam os *use cases* com as dependências e retornam a resposta. O `next.config.ts` configura CORS para requisições do frontend (`http://10.17.40.207:5173`) e define o PDFKit como pacote exclusivo do servidor.

Alternativas como Express e Fastify foram descartadas por exigirem configuração manual de middlewares, roteamento e TypeScript. O Next.js oferece essas funcionalidades de forma nativa (roteamento baseado em arquivos, TypeScript sem *tsconfig* adicional, validação de variáveis de ambiente via `next.config.ts`), além de simplificar o deploy com um único artefato *serverless*. O NestJS foi preterido por sua curva de aprendizado acentuada e pela sobrecarga de abstrações (decorators, módulos, injetores) que não se alinhavam à simplicidade almejada.

2.3.2 Supabase e PostgreSQL

Supabase é uma plataforma *open-source* que oferece PostgreSQL como serviço gerenciado, com APIs REST auto-geradas e autenticação integrada [18]. Diferentemente do Firebase (Google), que utiliza um banco NoSQL proprietário, o Supabase utiliza PostgreSQL, um banco relacional maduro que permite consultas complexas, integridade referencial e transações ACID.

A escolha do Supabase em vez de alternativas como Firebase foi motivada pelo modelo de dados relacional do sistema: certificados e templates possuem relações bem definidas (um template pode gerar muitos certificados) e exigem integridade referencial via chaves estrangeiras, algo que o Firestore (NoSQL do Firebase) não oferece nativamente, exigindo validações manuais no código da aplicação. O Appwrite, outra alternativa *open-source* BaaS, foi descartado por seu ecossistema menos maduro e pela ausência de um cliente oficial com suporte a SSR para Next.js no momento da decisão.

No sistema, o Supabase é utilizado exclusivamente como banco de dados relacional. As

consultas são feitas via `@supabase/supabase-js` e `@supabase/ssr`, sem ORMs como Prisma ou TypeORM. O PostgreSQL 16 [19] é o banco subjacente. A opção por SQL direto deu-se pelo controle fino sobre as consultas: operações como inserção em lote e CTEs são expressas de forma mais natural em SQL puro. O esquema contém duas tabelas principais: `templates` (coluna `layout_config` do tipo `JSONB`) e `certificates` (índices em `recipient_cpf` e `verification_code`). A nomenclatura segue *snake_case* no banco e *camelCase* no código, com mapeamento manual nos repositórios.

2.3.3 PDFKit

PDFKit é uma biblioteca Node.js para geração de documentos PDF no lado do servidor, sem dependência de ferramentas externas como wkhtmltopdf ou LaTeX [20]. Suporta texto com fontes embutidas, imagens, vetores e cores personalizadas.

A escolha do PDFKit em detrimento de alternativas como Puppeteer baseou-se em eficiência de recursos: o Puppeteer exige a execução de um navegador Chromium headless completo para rasterizar HTML em PDF, consumindo tipicamente mais de 100 MB de memória por instância. Em um ambiente serverless, onde cada requisição pode inicializar uma nova instância, esse custo inviabiliza o uso. O PDFKit opera exclusivamente em Node.js, com footprint de memória da ordem de dezenas de megabytes, e permite controle programático preciso sobre cada elemento do documento — posicionamento absoluto, fontes embutidas e cores arbitrárias — sem a intermediação de um motor de renderização HTML. Bibliotecas como jsPDF foram descartadas por operarem no lado do cliente (navegador), o que as tornaria inadequadas para um fluxo de geração server-side.

No sistema, o `PDFKitService` implementa a interface `IPDFService` definida no domínio, seguindo o princípio de inversão de dependência. O certificado é gerado em formato paisagem A4, com fundo colorido, bordas decorativas, título “CERTIFICADO”, nome do destinatário, curso, carga horária, datas de emissão e validade, CPF formatado e código de verificação gerado com UUID v4 [21]. Todas as cores, tamanhos e visibilidade dos elementos são controlados pelo `layoutConfig` do template, armazenado como `JSONB` no banco de dados. O PDF é retornado como `Buffer` e streamado diretamente na resposta HTTP.

2.3.4 Zod

Zod é uma biblioteca de validação de esquemas que, na versão 4, oferece inferência automática de tipos e composição de esquemas [15]. No backend, os esquemas são definidos em `application/dtos/index.ts` e utilizados nos *Route Handlers* para validar o corpo das requisições antes de repassá-los aos *use cases*. Essa validação centralizada garante que apenas dados consistentes cheguem à camada de domínio.

2.3.5 Arquitetura e Padrões

A arquitetura em camadas adotada no backend organiza o código em quatro pacotes: `domain`, `application`, `infrastructure` e `app`. O pacote `domain` contém as entidades e interfaces de repositório. O pacote `application` abriga os *use cases* de emissão, geração e verificação, além dos DTOs. O pacote `infrastructure` implementa os repositórios (`SupabaseCertificateRepository`, `SupabaseTemplateRepository`) e serviços (`PDFKitService`). O `app` contém os *Route Handlers*. Cada *use case* possui uma única responsabilidade, e as entidades encapsulam regras de negócio — `Certificate` valida a data antes de alterá-la.

2.4 Infraestrutura e Ferramentas

2.4.1 Render

Render é uma plataforma de cloud computing que oferece hospedagem simplificada para aplicações web, sites estáticos e serviços, com deploys automáticos a partir de repositórios Git e certificados SSL gratuitos [22].

O frontend, construído como uma SPA com Vite, é compilado para arquivos estáticos (HTML, CSS, JS) e implantado como um *Static Site* no Render. O deploy é acionado automaticamente a cada *push* na branch principal do GitHub. O arquivo `index.html` na raiz do projeto serve como ponto de entrada, e o roteamento no lado do cliente é gerenciado pelo TanStack Router, que utiliza *History API* para navegação sem recarregamento de página.

Alternativas como Vercel e Netlify foram avaliadas. O Render foi escolhido por dois motivos principais: (1) suporte nativo a sites estáticos com deploys automáticos a partir do GitHub, sem limite de largura de banda na camada gratuita, diferentemente da Vercel que impõe limites de banda em planos gratuitos; e (2) a possibilidade de hospedar tanto o frontend quanto o

backend na mesma plataforma, centralizando a gestão de infraestrutura em um único provedor — o frontend como *Static Site* e o backend como *Web Service*. O Cloudflare Pages também foi descartado por seu ecossistema de *Workers* focado em funções *edge*, modelo que não se aplica a um backend Node.js tradicional com dependências como PDFKit.

2.4.2 Jest

Jest é um framework de testes JavaScript desenvolvido pelo Facebook, que oferece um conjunto completo de funcionalidades: *runner* de testes, asserções, *mocking*, *code coverage* e *snapshot testing* [23]. Suporte a TypeScript é provido via `ts-jest`.

O Jest foi escolhido em detrimento do Vitest por dois motivos: (1) maturidade e estabilidade — o Jest possui mais de uma década de desenvolvimento, documentação extensa e uma comunidade consolidada, enquanto o Vitest, embora mais rápido por aproveitar o Vite, ainda estava em estágio inicial de adoção no momento da decisão do projeto; e (2) compatibilidade com o ecossistema — diversas bibliotecas do projeto (`@testing-library/react`, `jest-dom`, `ts-jest`) possuem integração madura e testada com o Jest, reduzindo o risco de incompatibilidades durante o desenvolvimento.

No sistema, o Jest está configurado com cobertura mínima de 75%, validada através da flag `--coverage` no pipeline de CI. Testes unitários foram escritos para as *ViewModels* e funções utilitárias, enquanto a configuração de *mocking* permite isolar as chamadas de API durante os testes sem depender de um servidor backend ativo.

2.4.3 ESLint e Prettier

ESLint é uma ferramenta de análise estática de código que identifica padrões problemáticos, erros de sintaxe e violações de boas práticas [24]. Prettier [25] é um formatador de código opinativo que garante consistência estilística em todo o projeto, eliminando discussões sobre espaçamento, aspas e ponto-e-vírgula.

No sistema, ambos foram configurados com plugins: `eslint-plugin-react-hooks` para hooks, `@typescript-eslint` para tipagem e `eslint-config-prettier` para evitar conflitos. A integração com o VS Code destaca erros em tempo real e formata o código ao salvar.

2.4.4 Git e GitHub

Git [26] é um sistema de controle de versão distribuído que permite rastrear alterações no código-fonte, gerenciar ramos de desenvolvimento paralelo e colaborar entre múltiplos desenvolvedores. GitHub [27] é a plataforma de hospedagem de repositórios que estende o Git com *Pull Requests*, *Issues*, *Actions* para CI/CD e revisão de código.

O fluxo de trabalho adotado foi o *Git Flow* simplificado, com as branches `main` (produção), `dev` (desenvolvimento) e `feature/*` (funcionalidades específicas). Cada funcionalidade é desenvolvida em uma branch separada e integrada à `dev` via *Pull Request*, que passa por revisão de código antes do merge.

2.4.5 Visual Studio Code

Visual Studio Code [28] é um editor de código-fonte desenvolvido pela Microsoft, baseado no Electron e no protocolo *Language Server Protocol* (LSP), que oferece suporte avançado a TypeScript, depuração integrada, terminal embutido e um ecossistema rico de extensões.

A produtividade foi ampliada com extensões como ESLint (análise estática), Prettier (formatação automática), Tailwind CSS IntelliSense (autocompletar para classes) e GitHub Copilot (sugestões contextuais).

2.5 Síntese das Tecnologias

A Tabela 1 apresenta um resumo das tecnologias discutidas neste capítulo, com suas respectivas finalidades no contexto do sistema.

Tabela 1 – Tecnologias utilizadas no desenvolvimento do sistema

Tecnologia	Versão	Finalidade
Frontend		
React	19	Construção da interface com componentes reutilizáveis
TypeScript	5.x	Tipagem estática e segurança em tempo de compilação
Tailwind CSS	4.x	Estilização utilitária com configuração CSS-first
shadcn/ui	~2.x	Componentes reutilizáveis baseados em Radix UI
TanStack Router	~1.x	Roteamento declarativo com proteção de rotas
React Hook Form	~7.x	Gerenciamento performático de formulários
Sonner	~2.x	Notificações toast acessíveis
Recharts	~2.x	Gráficos declarativos para dashboard
date-fns	~4.x	Utilitários de manipulação de datas
Vite	~7.x	Servidor de desenvolvimento e empacotamento
Zod	~3.x / 4.x	Validação de esquemas no frontend e backend
Backend		
Next.js	16	Framework para API REST com Route Handlers
Node.js	22 LTS	Ambiente de execução do servidor
Banco de Dados		
Supabase	~2.x	Plataforma de banco PostgreSQL gerenciado
PostgreSQL	16	Banco relacional com integridade referencial
Geração de Documentos		
PDFKit	~0.18	Geração server-side de certificados em PDF
UUID	~14.x	Geração de identificadores únicos para verificação
Infraestrutura e DevOps		
Render	SaaS	Hospedagem do frontend como site estático
Git	~2.x	Controle de versão distribuído
GitHub	SaaS	Repositório remoto e CI/CD via Actions
Ferramentas de Desenvolvimento		
Jest	~30.x	Testes unitários com cobertura mínima de 75%
ESLint	~9.x	Análise estática e linting de código
Prettier	~3.x	Formatador opinativo de código
Visual Studio Code	~1.x	Ambiente de desenvolvimento

3.0 Modelagem do Projeto

3.1 Levantamento de Requisitos

O levantamento de requisitos foi realizado por meio de uma entrevista semiestruturada com o responsável pela emissão de certificados na instituição parceira, juntamente com a observação direta do processo manual executado por ele. Durante a entrevista, foram identificadas as principais dificuldades enfrentadas, como o tempo elevado para emissão manual, a dificuldade em gerenciar grandes volumes de certificados e a ausência de um mecanismo de validação pública. A análise do processo permitiu mapear o fluxo completo de emissão, desde o cadastro de participantes até a entrega do certificado, servindo como base para a definição dos requisitos funcionais e não funcionais apresentados a seguir.

Requisitos Funcionais:

Os requisitos foram priorizados segundo o método MoSCoW, alinhado à abordagem MVP (Produto Mínimo Viável) adotada no projeto: **[M]** *Must have* — funcionalidades essenciais implementadas no MVP; **[S]** *Should have* — importantes, mas não críticas para a primeira versão; **[C]** *Could have* — desejáveis para versões futuras; **[W]** *Won't have* — fora do escopo do MVP.

3.1.1 Módulo de Autenticação e Controle de Acesso

- RF001 [M] – Permitir login por e-mail e senha.
- RF002 [S] – Permitir redefinição de senha via e-mail.
- RF003 [M] – Validar credenciais antes de conceder acesso.
- RF004 [S] – Permitir cadastro de novos usuários (apenas administrador).
- RF005 [M] – Permitir gerenciamento de perfis e permissões.
- RF006 [M] – Controlar acesso a áreas restritas com base no status de autenticação do usuário.
- RF007 [S] – Controlar sessões ativas por conta, permitindo apenas uma sessão simultânea.

3.1.2 Módulo de Templates de Certificados

- RF008 [M] – Permitir upload de templates (PPTX, PDF).
- RF009 [M] – Permitir personalização de templates.
- RF010 [M] – Permitir selecionar template da galeria interna.
- RF011 [C] – Identificar campos dinâmicos automaticamente.
- RF012 [M] – Permitir criação/edição manual de campos dinâmicos (ex: {{nome}}).
- RF013 [M] – Salvar configurações de mapeamento de campos.
- RF014 [C] – Permitir duplicar templates existentes.

3.1.3 Módulo de Entrada de Dados

- RF015 [M] – Permitir cadastro individual de participantes.
- RF016 [M] – Permitir emissão manual para participante único.
- RF017 [C] – Importar dados via CSV.
- RF018 [C] – Importar dados via Excel (.xlsx).
- RF019 [C] – Configurar gatilhos de emissão automática.
- RF020 [C] – Permitir upload de lista em lote.
- RF021 [C] – Processar múltiplos certificados em fila.
- RF022 [C] – Notificar ao término do processamento.

3.1.4 Módulo de Geração de Certificados

- RF023 [M] – Gerar certificados em PDF.
- RF024 [M] – Preencher automaticamente campos dinâmicos.
- RF025 [M] – Gerar código único por certificado.
- RF026 [M] – Registrar data, hora e localidade da emissão.

- RF027 [M] – Permitir reemissão mantendo histórico.
- RF028 [M] – Permitir download individual.
- RF029 [S] – Permitir download em lote.

3.1.5 Módulo de Envio e Comunicação

- RF030 [C] – Permitir personalização do corpo do e-mail com variáveis.
- RF031 [S] – Registrar status de envio (pendente, enviado, falha).
- RF032 [C] – Reenviar automaticamente certificados com falha de envio.

3.1.6 Módulo de Validação Pública e Antifraude

- RF033 [M] – Disponibilizar portal público de validação.
- RF034 [M] – Validar certificado via código único.
- RF035 [M] – Gerar QR Code no certificado com opções de personalização.
- RF036 [M] – Exibir informações básicas ao validar (nome, curso, data, validade, autenticidade).

3.1.7 Módulo de Assinaturas Digitais

- RF037 [C] – Permitir adicionar assinatura digitalizada ao template.
- RF038 [C] – Registrar hash de verificação da assinatura.

3.1.8 Módulo de Dashboard e Gestão

- RF039 [M] – Exibir lista completa de certificados emitidos.
- RF040 [M] – Permitir filtros por curso, período, status e instrutor.
- RF041 [M] – Exibir estatísticas mensais de emissão.
- RF042 [C] – Exibir logs de ações realizadas.

3.1.9 Módulo de Auditoria e Segurança Operacional

- RF043 [S] – Registrar responsável por cada certificado gerado.
- RF044 [S] – Registrar data, hora e local de ações relevantes.
- RF045 [S] – Permitir arquivamento de certificados como alternativa à exclusão definitiva.

3.1.10 Módulo de Gestão de Cursos/Eventos

- RF046 [M] – Permitir cadastro de cursos/eventos.
- RF047 [M] – Associar participantes ao curso.
- RF048 [M] – Definir carga horária.
- RF049 [S] – Registrar instrutor responsável.
- RF050 [M] – Configurar datas de início e término.
- RF051 [M] – Listar certificados vinculados ao curso.
- RF052 [M] – Validar vínculo do participante ao curso antes de permitir a emissão.
- RF053 [S] – Registrar local do curso.

3.1.11 Módulo de Participantes

- RF054 [M] – Cadastro completo de participantes (nome, e-mail, CPF opcional).
- RF055 [M] – Permitir edição de dados.
- RF056 [S] – Importar participantes sem duplicidade.
- RF057 [S] – Exibir histórico de certificados por participante.
- RF058 [M] – Permitir busca por nome, e-mail e CPF.

3.1.12 Módulo Multiempresa / Multi-tenant

- RF059 [C] – Permitir cadastro de organizações com dados institucionais (CNPJ, razão social, endereço e contato).
- RF060 [C] – Associar usuários cadastrados à sua respectiva organização.
- RF061 [C] – Permitir que cada organização gerencie seus próprios templates.
- RF062 [C] – Permitir que cada organização gerencie seus próprios usuários e perfis de acesso.

3.1.13 Módulo de Configurações do Sistema

- RF063 [C] – Personalizar mensagens padrão.
- RF064 [C] – Ativar/desativar envio automático.
- RF065 [C] – Configurar regras de reemissão.

3.1.14 Módulo de Armazenamento e Arquivos

- RF066 [M] – Armazenar templates no sistema.

3.1.15 Módulo de Notificações e Alertas

- RF067 [S] – Notificar falhas de emissão.
- RF068 [C] – Notificar sucesso no processamento de planilhas.
- RF069 [C] – Alertar quando template estiver incompleto ou inválido.

3.1.16 Módulo de Monitoramento e Logs

- RF070 [S] – Registrar logs de autenticação e tentativas de acesso.
- RF071 [C] – Registrar eventos críticos (falha geração PDF etc.).
- RF072 [C] – Permitir exportação de logs.

3.1.17 Módulo de Suporte, Operação e Manutenção

- RF073 [C] – Permitir abertura de chamados de suporte.
- RF074 [C] – Exibir status do sistema (online/manutenção).
- RF075 [C] – Permitir atualização de termos de uso e políticas.

Requisitos Não Funcionais:

3.1.18 Segurança

- RNF001 – O sistema deve utilizar o Supabase Auth para gerenciamento seguro de autenticação e armazenamento de senhas com hash criptográfico seguro (bcrypt).
- RNF002 – O sistema deve utilizar conexão segura HTTPS (TLS 1.2 ou superior).
- RNF003 – O sistema deve proteger dados sensíveis em repouso utilizando criptografia fornecida pela infraestrutura cloud.
- RNF004 – O sistema deve bloquear autenticação após 5 tentativas consecutivas inválidas (conforme política do provedor de autenticação).
- RNF005 – Sessões inativas devem expirar automaticamente após tempo configurável (ex.: 30 minutos).
- RNF006 – O sistema deve implementar controle de acesso baseado em papéis (RBAC).
- RNF007 – O sistema deve seguir boas práticas de segurança conforme OWASP Top 10.
- RNF008 – O sistema deve registrar logs de autenticação e eventos de segurança.
- RNF009 – O sistema deve garantir isolamento de dados entre empresas (multi-tenant).
- RNF010 – O sistema deve permitir futura implementação de autenticação multifator (MFA).
- RNF059 – O sistema deve permitir configurar horários de emissão por perfil de colaborador como política de segurança.

3.1.19 Desempenho e Escalabilidade

Os valores numéricos apresentados a seguir são estimativas definidas pelo desenvolvedor com base no porte esperado da operação, considerando turmas típicas de 20 a 50 participantes e emissões recorrentes ao longo do ano. Os limites estabelecidos consideram uma margem de segurança para absorver picos sazonais sem degradação da experiência do usuário.

- RNF011 – O sistema deve suportar geração simultânea de, estimados, 500 certificados por lote, volume equivalente a dez turmas médias processadas em única operação.
- RNF012 – O tempo médio de geração individual não deve exceder 5 segundos por certificado, garantindo que um lote de 500 certificados seja concluído em aproximadamente 40 minutos.
- RNF013 – O portal público de validação deve responder em até 2 segundos.
- RNF014 – O sistema deve permitir escalabilidade horizontal do serviço de geração.
- RNF015 – O sistema deve suportar volume estimado de 10.000 certificados/mês, compatível com a projeção de emissões recorrentes acrescida de margem de segurança.
- RNF016 – A geração em lote deve ser processada de forma assíncrona por meio de fila de processamento, sem bloquear a aplicação principal.
- RNF017 – A fila de certificados deve garantir processamento ordenado e controle de concorrência.

3.1.20 Disponibilidade e Confiabilidade

- RNF018 – O sistema deve garantir disponibilidade mínima de 99,5% mensal.
- RNF019 – O sistema deve realizar backup automático diário do banco de dados.
- RNF020 – O sistema deve permitir restauração de dados em até 24 horas.
- RNF021 – O sistema deve implementar mecanismo de retentativa automática para tarefas falhas na fila de certificados.
- RNF022 – O sistema deve registrar falhas de processamento para auditoria posterior.

3.1.21 Usabilidade e Acessibilidade

- RNF023 – O sistema deve ser responsivo (desktop, tablet e mobile).
- RNF024 – O sistema deve manter padrão visual consistente.
- RNF025 – O sistema deve fornecer mensagens de erro claras.
- RNF026 – O sistema deve permitir navegação por teclado.
- RNF027 – O sistema deve seguir recomendações WCAG 2.1 nível AA (quando aplicável).

3.1.22 Compatibilidade e Integração

- RNF028 – O sistema deve ser compatível com navegadores modernos (Chrome, Firefox, Edge, Safari).
- RNF029 – A geração de PDF deve preservar fidelidade visual do template original.
- RNF030 – O sistema deve disponibilizar API REST autenticada via token JWT do Supabase.
- RNF031 – O sistema deve aceitar arquivos CSV codificados em UTF-8.
- RNF032 – O sistema deve permitir integração via Webhooks em formato JSON.

3.1.23 Armazenamento e Retenção

- RNF033 – O sistema deve armazenar certificados por no mínimo 5 anos.
- RNF034 – O sistema deve garantir integridade dos certificados por meio de hash único.
- RNF035 – O sistema deve aplicar exclusão lógica (soft delete) quando aplicável.

3.1.24 Conformidade Legal e LGPD

- RNF036 – O sistema deve estar em conformidade com a LGPD (Lei nº 13.709/2018).
- RNF037 – O sistema deve permitir exclusão ou anonimização de dados pessoais mediante solicitação.

- RNF038 – O sistema deve registrar consentimento para tratamento de dados.
- RNF039 – Logs sensíveis devem ser acessíveis apenas por administradores autorizados.
- RNF040 – O sistema deve manter trilha de auditoria inviolável para operações críticas.

3.1.25 Arquitetura e Tecnologia

- RNF041 – O sistema deve ser composto por duas aplicações independentes: uma *Single Page Application* (SPA) para o frontend e uma API REST para o backend, cada uma com seu próprio ciclo de vida de desenvolvimento e deploy.
- RNF042 – O frontend deve adotar arquitetura em camadas com React, com quatro camadas: (i) *View* — componentes de interface; (ii) *ViewModel* — custom hooks que encapsulam estado e ações; (iii) *Service* — cliente de comunicação com a API; (iv) *Model* — interfaces TypeScript que definem as entidades de domínio.
- RNF043 – O backend deve adotar arquitetura em camadas (domínio, aplicação e infraestrutura), com as dependências apontando para a camada de domínio (princípio de inversão de dependências).
- RNF044 – A comunicação entre frontend e backend deve ocorrer exclusivamente via API REST sobre HTTPS, com autenticação baseada em token JWT.
- RNF045 – A geração de certificados em lote deve ser processada de forma assíncrona por meio de fila de processamento, sem bloquear a aplicação principal.
- RNF046 – O sistema deve utilizar banco de dados relacional com integridade referencial (PostgreSQL), gerenciado pelo Supabase.
- RNF047 – O sistema deve implementar estratégia de isolamento multi-tenant (por schema ou coluna organizacional).
- RNF048 – O sistema deve implementar monitoramento e observabilidade centralizada (logs, métricas e rastreamento).
- RNF049 – O frontend deve ser compilado como site estático e implantado em plataforma de hospedagem com deploy contínuo a partir do repositório Git.

- RNF050 – O backend deve ser implantado como serviço web independente em plataforma cloud compatível com Node.js.
- RNF051 – O sistema deve permitir extensibilidade, garantindo que novos módulos possam ser adicionados seguindo os mesmos padrões arquiteturais já estabelecidos.

3.1.26 Manutenção e Monitoramento

- RNF052 – O sistema deve permitir atualização com mínima indisponibilidade (deploy contínuo).
- RNF053 – O sistema deve possuir documentação técnica e manual do usuário.
- RNF054 – O sistema deve registrar erros em sistema centralizado de monitoramento.
- RNF055 – O sistema deve permitir extensibilidade sem impacto nas funcionalidades existentes.
- RNF060 – O sistema deve manter histórico de versões de templates e configurações para fins de auditoria e rastreabilidade.

3.1.27 Internacionalização

- RNF056 – O sistema deve permitir configuração de timezone por organização.
- RNF057 – O sistema deve permitir tradução futura da interface.
- RNF058 – O sistema deve suportar diferentes formatos de data e hora.

3.2 Regras de Negócio

Além dos requisitos funcionais, foram identificadas regras de negócio que impõem restrições ou condições sobre as funcionalidades do sistema:

- RN001 – Não deve ser permitido acesso a áreas restritas sem autenticação prévia.
- RN002 – Não deve ser permitido acesso simultâneo com a mesma conta.
- RN003 – Certificados não podem ser excluídos definitivamente, devendo ser arquivados.

- RN004 – Certificados só podem ser emitidos para participantes previamente vinculados ao curso ou evento.
- RN006 – O reenvio automático de certificados com falha deve respeitar o limite máximo de 3 tentativas com intervalo mínimo de 10 minutos entre cada uma.

3.3 Critérios de Aceitação

Cada requisito funcional possui critérios de aceitação que definem as condições necessárias para considerá-lo atendido. A Tabela 2 apresenta os critérios para todos os RFs.

Tabela 2 – Critérios de aceitação dos requisitos funcionais

RF	Descrição	Critério de Aceitação
RF001	Login por e-mail e senha	Usuário com credenciais válidas acessa o sistema; credenciais inválidas exibem mensagem de erro sem redirecionar.
RF002	Redefinição de senha	Usuário clica em "Esqueci senha", insere e-mail e recebe link de redefinição válido por 1 hora.
RF003	Validar credenciais	Rotas protegidas redirecionam para login quando o usuário não está autenticado; API retorna 401 para requisições sem token.
RF004	Cadastro de usuários	Apenas administrador pode acessar tela de cadastro; formulário exige nome, e-mail e senha; usuário criado aparece na lista.
RF005	Gerenciar perfis	Administrador pode listar, criar, editar e desativar usuários; alterações persistem após recarregar.
RF006	Controle de acesso	Usuário não autenticado é redirecionado para login ao acessar qualquer rota administrativa.
RF007	Sessão única	Login em novo dispositivo invalida sessão anterior; sessão expira após 30 min de inatividade.
RF008	Upload de templates	Arquivos .pptx e .pdf são aceitos; formatos .exe, .zip são rejeitados com mensagem clara; template aparece na galeria após upload.
RF009	Personalização	Cores, fontes e bordas configuradas são refletidas no preview visual.
RF010	Selecionar template	Galeria exibe templates salvos com nome e preview; seleção carrega o template para edição.
RF011	Identificar campos automáticos	Sistema sugere campos {{nome}}, {{curso}} ao fazer upload; sugestões são editáveis.
RF012	Criar/editar campos	Usuário adiciona campo {{cpf}}, salva, e o campo aparece na tela de emissão.

RF	Descrição	Critério de Aceitação
RF013	Salvar configurações	Mapeamento de campos persiste após recarregar a página.
RF014	Duplicar templates	Duplicata cria cópia idêntica com nome "(cópia)"; original não é alterado.
RF015	Cadastro individual	Formulário válido salva participante; e-mail duplicado exibe alerta; participante aparece na lista.
RF016	Emissão manual	Selecionar template, preencher dados e confirmar gera certificado listado no dashboard.
RF017	Importar CSV	Arquivo CSV com cabeçalho correto importa participantes; linhas com erro são ignoradas e reportadas.
RF018	Importar Excel	Arquivo .xlsx com abas é processado; primeira aba é usada como padrão.
RF019	Gatilhos automáticos	Ao cadastrar participante em curso com gatilho ativo, certificado é emitido automaticamente em até 30s.
RF020	Upload em lote	Arquivo com 100 participantes é processado; certificados são gerados para todos.
RF021	Fila de processamento	Lote de 500 participantes é enfileirado; barra de progresso mostra andamento.
RF022	Notificar término	Processamento concluído envia notificação na tela (toast) com resumo.
RF023	Gerar PDF	PDF gerado preserva layout, cores e fontes do template; campos preenchidos aparecem nas posições corretas; geração leva menos de 5 segundos.
RF024	Preencher campos	Dados do participante substituem corretamente os {{placeholders}} no PDF.
RF025	Código único	Cada certificado possui código único (UUID v4); consulta no banco retorna apenas um registro.
RF026	Data/hora/local	Certificado exibe data e hora da emissão; localização é definida pelo curso ou configuração.
RF027	Reemissão	Reemissão gera novo PDF com novo código; histórico lista versão anterior com data.
RF028	Download individual	PDF é baixado com nome no formato Nome_Curso_Data.pdf.
RF029	Download em lote	Selecionar múltiplos certificados e baixar gera arquivo .zip com todos os PDFs.

RF	Descrição	Critério de Aceitação
RF030	Personalizar e-mail	Variáveis {{nome}}, {{link}} são substituídas no corpo do e-mail enviado.
RF031	Status de envio	Certificado exibe status "Pendente", "Enviado" ou "Falha"; status é atualizado em tempo real.
RF032	Reenvio automático	Falha de envio dispara nova tentativa após 10 min, até 3 tentativas.
RF033	Portal de validação	URL pública /validar exibe campo para inserir código; página é responsiva.
RF034	Validar via código	Código válido exibe dados do certificado; inválido mostra "Certificado não encontrado".
RF035	QR Code	QR Code no PDF redireciona para URL de validação com código como parâmetro.
RF036	Exibir informações	Tela de validação mostra nome, curso, data, carga horária e status "Autêntico".
RF037	Assinatura digitalizada	Imagem de assinatura aparece na posição configurada no template.
RF038	Hash da assinatura	Hash SHA-256 da assinatura é armazenado e verificável.
RF039	Lista de certificados	Tabela exibe certificados com colunas: nome, curso, data, status; paginação de 20 itens.
RF040	Filtros	Filtrar por curso, período, status e instrutor atualiza a tabela sem recarregar a página.
RF041	Estatísticas mensais	Dashboard exibe total de emissões por mês em gráfico de barras; dados são atualizados automaticamente.
RF042	Logs de ações	Tabela de logs exibe ação, usuário, data e detalhes; ordenada por data decrescente.
RF043	Responsável	Certificado gerado registra ID do usuário que emitiu; exibido no histórico.
RF044	Data/hora ações	Toda ação de emissão, edição e exclusão registra timestamp.
RF045	Arquivamento	Certificado arquivado não aparece na lista padrão, mas é recuperável via filtro "Arquivados".
RF046	Cadastro cursos	Formulário salva nome, tipo, carga horária, datas e instrutor; curso aparece na lista.
RF047	Associar participantes	Participante é vinculado ao curso no momento da emissão.

RF	Descrição	Critério de Aceitação
RF048	Carga horária	Carga horária definida no curso aparece no campo correspondente do certificado.
RF049	Instrutor	Nome do instrutor é salvo e exibido no certificado.
RF050	Configurar datas	Datas configuradas são preenchidas automaticamente no certificado.
RF051	Listar por curso	Filtro "Curso"exibe apenas certificados daquele curso.
RF052	Validar vínculo	Emissão é bloqueada se participante não pertence ao curso selecionado; mensagem exibe motivo.
RF053	Registrar local	Local do curso é salvo e exibido no certificado.
RF054	Cadastro completo	Nome e e-mail são obrigatórios; CPF é opcional; e-mail inválido é rejeitado.
RF055	Edição	Dados editados são salvos e refletidos na tela de emissão.
RF056	Importar sem duplicidade	Mesmo CPF/e-mail não cria registro duplicado; dados existentes são atualizados.
RF057	Histórico	Página do participante exibe todos os certificados emitidos para ele.
RF058	Busca	Busca por nome retorna resultados em menos de 2s; busca vazia retorna lista completa.
RF059	Cadastro organizações	Formulário salva CNPJ, razão social e contato; organização aparece na lista.
RF060	Associar user/org	Usuário é vinculado a uma organização no cadastro; vínculo é editável.
RF061	Templates por org	Organização A não vê templates da organização B.
RF062	Usuários por org	Organização A não vê usuários da organização B.
RF063	Mensagens padrão	Texto padrão de e-mail é configurável por organização.
RF064	Envio automático	Opção "Enviar automaticamente"é ativável/desativável por organização.
RF065	Regras de reemissão	Reemissão é permitida apenas se configurada; limite de reemissões é respeitado.
RF066	Armazenar templates	Template enviado fica imediatamente disponível para uso; download do template original é possível.
RF067	Notificar falhas	Falha de emissão dispara notificação ao administrador com detalhes do erro.
RF068	Notificar sucesso	Processamento concluído com sucesso exibe toast com total de certificados.

RF	Descrição	Critério de Aceitação
RF069	Alertar template inválido	Template sem campos {{nome}} exibe alerta antes da emissão.
RF070	Logs de autenticação	Tentativa de login com senha incorreta é registrada com timestamp e IP.
RF071	Eventos críticos	Falha de geração de PDF é registrada com stack trace e ID do certificado.
RF072	Exportar logs	Logs são exportáveis em CSV com filtro por período.
RF073	Chamados de suporte	Formulário de chamado salva descrição, prioridade e contato; chamado é listado.
RF074	Status do sistema	Página de status exibe "Online", "Em manutenção" ou "Indisponível".
RF075	Termos de uso	Atualização de termos exibe aviso para aceite na próxima autenticação.

3.4 Matriz de Rastreabilidade

A Tabela 3 apresenta a matriz de rastreabilidade que relaciona os requisitos funcionais aos casos de uso, às principais classes do diagrama de classes e às entidades do banco de dados, garantindo a rastreabilidade entre os artefatos do projeto.

Módulo / RFs	Casos de Uso	Classes	Tabelas
Autenticação e Controle de Acesso (RF001–RF007)	Gerenciar Usuários, Definir Permissões	Usuario, UsuarioOrganizacao, Sessao	USUARIO, USUARIO_ORGANIZACAO, SESSAO
Templates de Certificados (RF008–RF014)	Criar Template, Editar Template, Configurar Campos, Salvar, Prévia	Template, CampoDinamico, TemplateVersao	TEMPLATE, CAMPO_DINAMICO, TEMPLATE_VERSAO
Entrada de Dados (RF015–RF022)	Cadastrar Part., Importar Part., Emitir Individual, Emitir em Lote	Participante, Importacao	PESSOA, PARTICIPANTE, IMPORTACAO
Geração de Certificados (RF023–RF029)	Emitir Individual, Emitir em Lote, Fila, Notificar	Certificado, ValorCampo, FilaProcessamento	CERTIFICADO, VALOR_CAMPO, FILA_PROCESSAMENTO
Envio e Comunicação (RF030–RF032)	Notificar	EnvioEmail	ENVIO_EMAIL
Validação Pública e Antifraude (RF033–RF036)	Validar Cert., Exibir Dados, Escanear QR Code	Certificado	CERTIFICADO
Assinaturas Digitais (RF037–RF038)	Emitir Individual	AssinaturaDigital	ASSINATURA_DIGITAL
Dashboard e Gestão (RF039–RF042)	—	—	views / consultas SQL
Auditoria e Segurança (RF043–RF045)	—	LogSistema, LogAuditoria	LOGSISTEMA, LOG_AUDITORIA
Gestão de Cursos/Eventos (RF046–RF053)	Gerenciar Cursos	Curso, Matricula	CURSO, MATRICULA
Participantes (RF054–RF058)	Cadastrar Part., Importar Part., Buscar Part., Visualizar Histórico	Participante	PESSOA, PARTICIPANTE
Multiempresa (RF059–RF062)	Gerenciar Organizações	Organizacao, UsuarioOrganizacao	ORGANIZACAO, USUARIO_ORGANIZACAO
Configurações do Sistema (RF063–RF065)	Configurar Sistema	ConfiguracaoSistema	CONFIGURACAO_SISTEMA
Armazenamento e Arquivos (RF066)	Criar Template, Editar Template	Template, TemplateVersao	TEMPLATE, TEMPLATE_VERSAO
Notificações e Alertas (RF067–RF069)	Notificar	EnvioEmail	ENVIO_EMAIL
Monitoramento e Logs (RF070–RF072)	—	LogSistema, LogAuditoria	LOGSISTEMA, LOG_AUDITORIA
Suporte e Operação (RF073–RF075)	Abrir Chamado	ChamadoSuporte	CHAMADO_SUORTE

Tabela 3 – Matriz de rastreabilidade entre requisitos funcionais, casos de uso, classes e tabelas do banco de dados

3.5 Diagramas de casos de uso

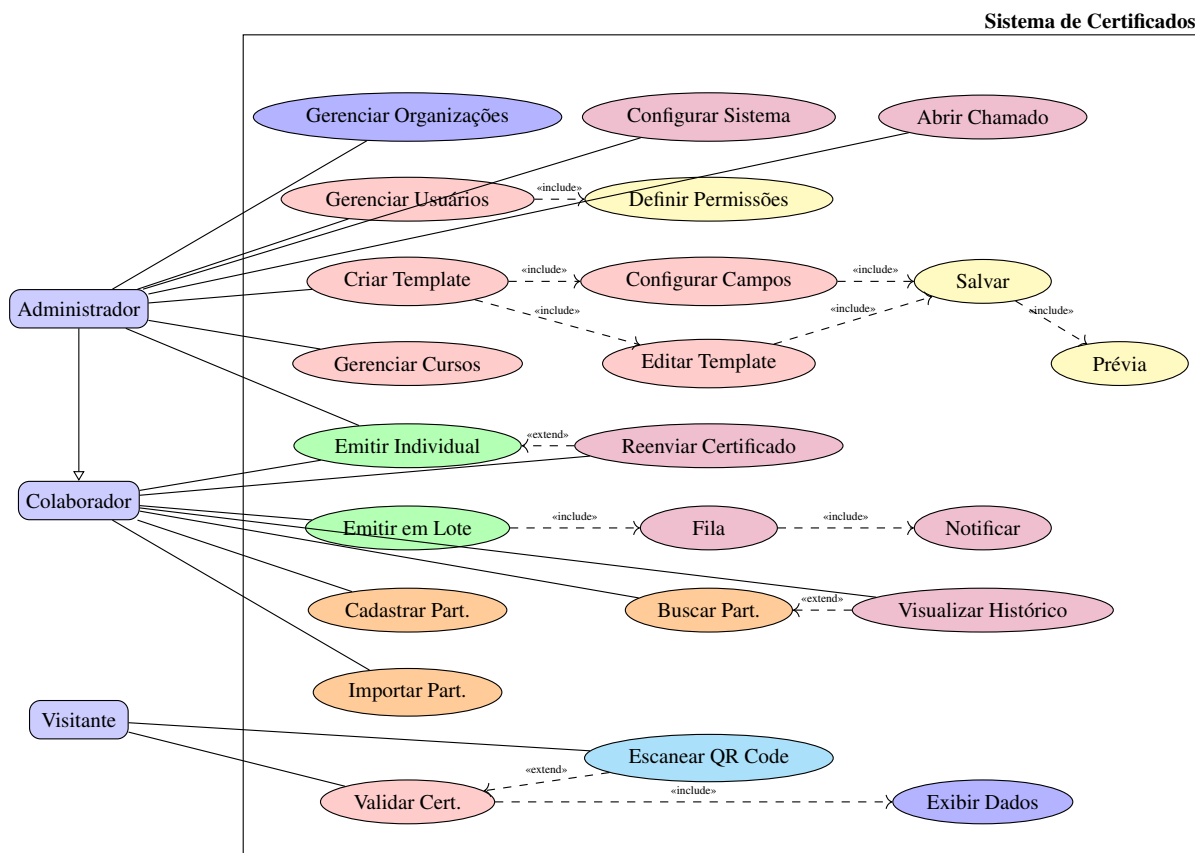


Figura 1 – Diagrama de casos de uso do sistema de certificados

As descrições textuais a seguir detalham cada caso de uso identificado no diagrama da Figura 1, seguindo o formato padronizado proposto por Krämer [29].

UC001	Gerenciar Usuários	
Dependências	RF001, RF003, RF004, RF005, RF006	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o administrador gerencie os usuários da plataforma, incluindo criação, edição e desativação de contas, bem como a definição de perfis de acesso.	
Ator(es)	Administrador	
Pré-condições	1. Administrador autenticado no sistema. 2. Administrador possui permissão de gerenciamento de usuários.	
Sequência Ordinária	Passo	Ação
	1	O administrador acessa a seção de usuários.
	2	O sistema exibe a lista de usuários cadastrados.
	3	O administrador seleciona criar, editar ou desativar um usuário.
	4	O sistema valida os dados informados.
	5	O sistema persiste a alteração no banco de dados.
	6	O sistema exibe confirmação da operação.
Pós-condições	1. Usuário criado, editado ou desativado com sucesso. 2. Alterações refletidas na lista de usuários.	
Exceções	Passo	Condição
	3	Se o e-mail informado já existir, o sistema rejeita o cadastro e exibe mensagem de erro.
Referências	RF001, RF003, RF004, RF005, RF006	
Comentários	Nenhum.	

Tabela 4 – Caso de uso UC001 – Gerenciar Usuários

UC002	Gerenciar Templates	
Dependências	RF008, RF009, RF010, RF012, RF013	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o administrador gerencie templates de certificados, incluindo upload, personalização visual e configuração de campos dinâmicos.	
Ator(es)	Administrador	
Pré-condições	1. Administrador autenticado no sistema.	
Sequência Ordinária	Passo	Ação
	1	O administrador acessa a seção de templates.
	2	O sistema exibe a galeria de templates existentes.
	3	O administrador faz upload de um novo arquivo PPTX ou PDF.
	4	O sistema valida o formato do arquivo e exibe preview.
	5	O administrador configura campos dinâmicos e personaliza cores e fontes.
	6	O sistema salva o template e retorna à galeria.
Pós-condições	1. Template salvo no sistema. 2. Template disponível para uso na emissão de certificados.	
Exceções	Passo	Condição
	3	Se o arquivo enviado não for PPTX ou PDF, o sistema rejeita e exibe mensagem de formato inválido.
Referências	RF008, RF009, RF010, RF012, RF013	
Comentários	Nenhum.	

Tabela 5 – Caso de uso UC002 – Gerenciar Templates

UC003	Gerenciar Cursos	
Dependências	RF046, RF047, RF048, RF049, RF050, RF053	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o administrador cadastre e gerencie cursos, incluindo nome, carga horária, datas, local e instrutor.	
Ator(es)	Administrador	
Pré-condições	1. Administrador autenticado no sistema.	
Sequência Ordinária	Passo	Ação
	1	O administrador acessa a seção de cursos.
	2	O sistema exibe a lista de cursos cadastrados.
	3	O administrador preenche nome, carga horária, datas e instrutor.
	4	O sistema valida os dados obrigatórios.
	5	O sistema salva o curso no banco de dados.
	6	O sistema exibe confirmação.
Pós-condições	1. Curso cadastrado no sistema. 2. Curso disponível para vinculação com certificados.	
Exceções	Passo	Condição
	4	Se dados obrigatórios estiverem ausentes, o sistema bloqueia o salvamento e exibe os campos pendentes.
Referências	RF046, RF047, RF048, RF049, RF050, RF053	
Comentários	Nenhum.	

Tabela 6 – Caso de uso UC003 – Gerenciar Cursos

UC004	Emitir Certificado Individual	
Dependências	RF015, RF016, RF023, RF024, RF025, RF026, RF028, RF052	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o colaborador emita um certificado individual, selecionando template, curso e participante, e gerando PDF com código único e QR Code.	
Ator(es)	Colaborador	
Pré-condições	1. Colaborador autenticado no sistema. 2. Template de certificado cadastrado. 3. Curso cadastrado.	
Sequência Ordinária	Passo	Ação
	1	O colaborador acessa a seção de emissão.
	2	O sistema exibe formulário com campos para template, curso e participante.
	3	O colaborador seleciona o template e o curso.
	4	O colaborador seleciona ou cadastra o participante.
	5	O sistema valida o vínculo do participante ao curso.
	6	O sistema gera o PDF preenchendo os campos dinâmicos.
	7	O sistema gera código único e insere QR Code no PDF.
	8	O sistema registra data, hora e responsável pela emissão.
	9	O sistema exibe o certificado gerado com opção de download.
Pós-condições	1. Certificado gerado em PDF. 2. Código único registrado no banco de dados. 3. Certificado disponível para validação pública.	
Exceções	Passo	Condição
	5	Se o participante não estiver vinculado ao curso, o sistema bloqueia a emissão e exibe alerta.
Referências	RF015, RF016, RF023, RF024, RF025, RF026, RF028, RF052	
Comentários	Nenhum.	

Tabela 7 – Caso de uso UC004 – Emitir Certificado Individual

UC005	Emitir Certificados em Lote	
Dependências	RF017, RF020, RF021, RF022	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o colaborador emita certificados em lote a partir de uma lista de participantes, processando de forma assíncrona por fila.	
Ator(es)	Colaborador	
Pré-condições	1. Colaborador autenticado no sistema. 2. Template de certificado cadastrado. 3. Arquivo com lista de participantes disponível.	
Sequência Ordinária	Passo	Ação
	1	O colaborador acessa a seção de emissão em lote.
	2	O sistema exibe formulário para seleção de template e arquivo.
	3	O colaborador seleciona o template e envia o arquivo.
	4	O sistema valida os dados do arquivo.
	5	O sistema enfileira as emissões para processamento.
	6	O sistema processa cada item da fila, gerando PDF e código único.
	7	O sistema notifica o colaborador ao término do processamento.
Pós-condições	1. Certificados do lote gerados e registrados. 2. Colaborador notificado sobre o resultado.	
Exceções	Passo	Condição
	4	Se o arquivo tiver formato inválido, o sistema rejeita e exibe mensagem.
	6	Se alguma linha contiver dados inválidos, o sistema ignora e reporta ao final.
Referências	RF017, RF020, RF021, RF022	
Comentários	Nenhum.	

Tabela 8 – Caso de uso UC005 – Emitir Certificados em Lote

UC006	Cadastrar Participante	
Dependências	RF015, RF054, RF055, RF058	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o colaborador cadastre participantes individualmente, informando nome, e-mail e CPF opcional.	
Ator(es)	Colaborador	
Pré-condições	1. Colaborador autenticado no sistema.	
Sequência Ordinária	Passo	Ação
	1	O colaborador acessa a seção de participantes.
	2	O sistema exibe formulário com campos de nome, e-mail e CPF.
	3	O colaborador preenche os dados e confirma.
	4	O sistema valida o formato do e-mail.
	5	O sistema salva o participante no banco de dados.
	6	O sistema exibe confirmação.
Pós-condições	1. Participante cadastrado no sistema. 2. Participante disponível para emissão de certificados.	
Exceções	Passo	Condição
	4	Se o e-mail já estiver cadastrado, o sistema alerta sobre registro existente.
	4	Se o formato do e-mail for inválido, o sistema rejeita e exibe mensagem.
Referências	RF015, RF054, RF055, RF058	
Comentários	Nenhum.	

Tabela 9 – Caso de uso UC006 – Cadastrar Participante

UC007	Importar Participantes	
Dependências	RF017, RF018, RF056	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o colaborador importe participantes em lote a partir de arquivos CSV ou Excel.	
Ator(es)	Colaborador	
Pré-condições	1. Colaborador autenticado no sistema. 2. Arquivo CSV ou Excel disponível.	
Sequência Ordinária	Passo	Ação
	1	O colaborador acessa a opção de importar participantes.
	2	O sistema exibe formulário para upload de arquivo.
	3	O colaborador seleciona e envia o arquivo.
	4	O sistema valida o cabeçalho e os dados das linhas.
	5	O sistema importa os registros válidos.
	6	O sistema exibe resumo com quantidade de registros importados e rejeitados.
Pós-condições	1. Participantes válidos importados e disponíveis para emissão. 2. Relatório de erros gerado para registros rejeitados.	
Exceções	Passo	Condição
	4	Se o formato do arquivo não for CSV ou XLSX, o sistema rejeita e exibe mensagem.
	4	Se alguma linha contiver dados inconsistentes, o sistema ignora e lista ao final.
Referências	RF017, RF018, RF056	
Comentários	Nenhum.	

Tabela 10 – Caso de uso UC007 – Importar Participantes

UC008	Validar Certificado	
Dependências	RF033, RF034, RF035, RF036	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve disponibilizar um portal público onde qualquer visitante possa validar um certificado informando o código único ou escaneando o QR Code.	
Ator(es)	Visitante	
Pré-condições	1. Nenhuma (portal público, sem autenticação).	
Sequência Ordinária	Passo	Ação
	1	O visitante acessa o portal público de validação.
	2	O sistema exibe campo para inserção do código único.
	3	O visitante insere o código presente no certificado.
	4	O sistema consulta o código no banco de dados.
	5	O sistema exibe nome, curso, data e status de autenticidade.
Pós-condições	1. Informações do certificado exibidas ao visitante.	
Exceções	Passo	Condição
	3	Se o visitante escanear o QR Code, o sistema redireciona com o código preenchido automaticamente.
	4	Se o código não existir, o sistema exibe "Certificado não encontrado".
Referências	RF033, RF034, RF035, RF036	
Comentários	A validação via QR Code elimina a necessidade de digitação manual do código.	

Tabela 11 – Caso de uso UC008 – Validar Certificado

UC009	Buscar Participantes	
Dependências	RF058	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o colaborador busque participantes por nome, e-mail ou CPF, exibindo os resultados de forma rápida e paginada.	
Ator(es)	Colaborador	
Pré-condições	1. Colaborador autenticado no sistema.	
Sequência Ordinária	Passo	Ação
	1	O colaborador acessa a seção de participantes.
	2	O sistema exibe campo de busca e lista completa de participantes.
	3	O colaborador digita nome, e-mail ou CPF e confirma.
	4	O sistema filtra os registros correspondentes.
	5	O sistema exibe os resultados em menos de 2 segundos.
Pós-condições	1. Lista de participantes filtrada exibida ao colaborador.	
Exceções	Passo	Condição
	4	Se nenhum registro corresponder à busca, o sistema exibe mensagem "Nenhum participante encontrado".
	4	Se a busca estiver vazia, o sistema retorna a lista completa.
Referências	RF058	
Comentários	Nenhum.	

Tabela 12 – Caso de uso UC009 – Buscar Participantes

UC010	Visualizar Histórico de Certificados	
Dependências	RF057, RF058	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve exibir o histórico completo de certificados emitidos para um participante específico, incluindo data de emissão, template utilizado e status de validade.	
Ator(es)	Colaborador	
Pré-condições	1. Colaborador autenticado no sistema. 2. Participante localizado via busca.	
Sequência Ordinária	Passo	Ação
	1	O colaborador acessa a página do participante.
	2	O sistema exibe os dados cadastrais do participante.
	3	O colaborador seleciona a opção "Histórico de Certificados".
	4	O sistema consulta todos os certificados associados ao participante.
	5	O sistema exibe lista com data, template, curso e status de cada certificado.
Pós-condições	1. Histórico de certificados exibido ao colaborador.	
Exceções	Passo	Condição
	4	Se o participante não possuir certificados, o sistema exibe "Nenhum certificado encontrado".
Referências	RF057, RF058	
Comentários	Estende o caso de uso Buscar Participantes.	

Tabela 13 – Caso de uso UC010 – Visualizar Histórico de Certificados

UC011	Gerenciar Organizações	
Dependências	RF059, RF060, RF061, RF062	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o administrador cadastre e gerencie organizações (multi-tenant), incluindo CNPJ, razão social, endereço e contato, bem como a associação de usuários e templates a cada organização.	
Ator(es)	Administrador	
Pré-condições	1. Administrador autenticado no sistema.	
Sequência Ordinária	Passo	Ação
	1	O administrador acessa a seção de organizações.
	2	O sistema exibe a lista de organizações cadastradas.
	3	O administrador seleciona criar, editar ou desativar uma organização.
	4	O administrador preenche CNPJ, razão social, endereço e contato.
	5	O sistema valida o CNPJ e os dados obrigatórios.
	6	O sistema persiste a organização no banco de dados.
	7	O sistema exibe confirmação da operação.
Pós-condições	1. Organização cadastrada, editada ou desativada com sucesso. 2. Organização disponível para associação de usuários e templates.	
Exceções	Passo	Condição
	5	Se o CNPJ já estiver cadastrado, o sistema rejeita e exibe mensagem de duplicidade.
	5	Se o CNPJ for inválido, o sistema rejeita e exibe mensagem.
Referências	RF059, RF060, RF061, RF062	
Comentários	Nenhum.	

Tabela 14 – Caso de uso UC011 – Gerenciar Organizações

UC012	Configurar Sistema	
Dependências	RF063, RF064, RF065	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o administrador configure parâmetros gerais do sistema, incluindo personalização de mensagens padrão, ativação de envio automático e regras de reemissão de certificados.	
Ator(es)	Administrador	
Pré-condições	1. Administrador autenticado no sistema.	
Sequência Ordinária	Passo	Ação
	1	O administrador acessa a seção de configurações.
	2	O sistema exibe os painéis de configuração disponíveis.
	3	O administrador personaliza mensagens padrão de e-mail.
	4	O administrador ativa ou desativa o envio automático.
	5	O administrador configura limites e regras de reemissão.
	6	O sistema valida e persiste as configurações.
	7	O sistema exibe confirmação da operação.
Pós-condições	1. Configurações do sistema atualizadas. 2. Novas regras e mensagens aplicadas às próximas operações.	
Exceções	Passo	Condição
	6	Se algum valor obrigatório estiver ausente, o sistema bloqueia o salvamento e exibe os campos pendentes.
Referências	RF063, RF064, RF065	
Comentários	Nenhum.	

Tabela 15 – Caso de uso UC012 – Configurar Sistema

UC013	Abrir Chamado de Suporte	
Dependências	RF073	
Autor	Francisco Igor Silva Santos	
Descrição	O sistema deve permitir que o administrador abra chamados de suporte técnico, informando descrição do problema, prioridade e dados de contato.	
Ator(es)	Administrador	
Pré-condições	1. Administrador autenticado no sistema.	
Sequência Ordinária	Passo	Ação
	1	O administrador acessa a seção de suporte.
	2	O sistema exibe o formulário de abertura de chamado.
	3	O administrador preenche descrição, prioridade e contato.
	4	O sistema valida os dados obrigatórios.
	5	O sistema registra o chamado no banco de dados.
	6	O sistema exibe confirmação com número do chamado.
Pós-condições	1. Chamado registrado no sistema. 2. Chamado disponível para acompanhamento pela equipe de suporte.	
Exceções	Passo	Condição
	4	Se campos obrigatórios estiverem ausentes, o sistema bloqueia o envio e exibe os campos pendentes.
Referências	RF073	
Comentários	Nenhum.	

Tabela 16 – Caso de uso UC013 – Abrir Chamado de Suporte

3.6 Diagramas de classe

O diagrama de classes da Figura 2 modela as entidades de domínio do sistema de certificados com 21 classes distribuídas em módulos funcionais que refletem a separação de responsabilidades definida nos requisitos. Cada classe é identificada por um UUID único (com exceção de Sessao, que utiliza tokenHash), garantindo escalabilidade e independência entre ambientes. Este diagrama corresponde à camada de domínio (*domain layer*) da arquitetura em camadas adotada — as camadas de aplicação (*use cases*, DTOs) e infraestrutura (*repositories*, serviços externos) são representadas no diagrama de arquitetura da Figura 3.

A hierarquia de herança tem como base a classe abstrata Pessoa, que concentra os atributos comuns de identificação (id, nome, email) e o status ativo. Dela derivam Usuario e Participante.

Usuario representa colaboradores e administradores, com controle de perfil (`PerfilEnum`), `ultimoLogin` e restrição de horários permitidos (`horarioPermitidoInicio`, `horarioPermitidoFim`), além dos métodos públicos `autenticar(email, senha)` e `emitirCertificado(participante, template)`. Participante armazena dados cadastrais (`cpf`, `dataCadastro`), com o método `vincularCurso(curso)`. Essa generalização segue o princípio de herança da orientação a objetos, evitando duplicação de atributos e centralizando a lógica comum de pessoas.

O modelo multi-tenant é implementado pela classe `Organizacao` (`nome`, `cnpj`, `ativo`), que se associa em cardinalidade 1 para 0..* a `Pessoa` (da qual derivam `Usuario` e `Participante`), `Curso` (`nome`, `cargaHoraria`, `descricao`), `Template` e `Certificado`, e em 1 para 1 com `ConfiguracaoSistema`, além de 1 para 0..* com `ChamadoSuporte`, expondo o método `adicionarMembro(usuario)`. Cada organização opera de forma isolada, garantindo que os dados de um inquilino não sejam acessíveis por outro. A relação com a hierarquia de `Pessoa` se justifica porque a organização pode cadastrar participantes e usuários manualmente, independentemente de matrícula em curso específico. `Curso` oferece `inscreverParticipante(participante)` para matricular alunos.

A relação entre `Participante` e `Curso` é muitos-para-muitos (0..* para 0..*), materializada pela classe associativa `Matricula` (renomeada de `ParticipanteCurso` para alinhamento com o DER), que armazena `dataVinculo` e `status` (`StatusMatriculaEnum`: `ativo`, `concluído`, `cancelado`, `trancado`). Essa modelagem foi escolhida por permitir rastrear historicamente quando um participante se vinculou a cada curso e o estado atual do vínculo, além de facilitar consultas futuras como “alunos ativos por período”.

O vínculo entre `Usuario` e `Organizacao` é modelado pela classe `UsuarioOrganizacao`, que implementa a relação muitos-para-muitos com o atributo `perfil` (`PerfilEnum`). Diferentemente da abordagem de incluir `organizacao_id` diretamente nas tabelas de dados (usada no DER para isolamento multi-tenant), a classe `UsuarioOrganizacao` gerencia exclusivamente o controle de acesso, permitindo que um mesmo usuário atue em múltiplas organizações com perfis distintos sem duplicar dados cadastrais.

No módulo de templates, a classe `Template` armazena `nome` e `ativo`. O versionamento é tratado pela classe `TemplateVersao` (`versao`, `arquivoPath`, `criadoEm`), que se relaciona em 1 para 0..* com `Template`, garantindo que certificados emitidos anteriormente mantenham o layout original. `CampoDinamico` — com `nomeCampo`, `marcador` e `tipo` — modela os campos editáveis definidos em cada template, em composição 1 para 0..*. A assinatura digital é representada por `AssinaturaDigital` (`imagemAssinatura`, `hashVerificacao`, `dataRegistro`), opcionalmente vin-

culada ao template (1 para 0..1). Essa assinatura do template é independente da assinatura do certificado — servem a propósitos distintos: a do template é uma referência visual opcional do modelo, enquanto a do certificado é a assinatura efetiva que valida o documento emitido. Os valores preenchidos nos campos dinâmicos durante a emissão são armazenados por Valor-Campo (id, valor) em associação 1 para 0..* com Certificado, referenciando CampoDinamico para definir qual campo foi preenchido.

A classe Certificado é a entidade central do sistema, associada a Organizacao, ao Usuario que o emitiu (relação nomeada “emitiu”), a Participante (0..* para 0..*), a Curso (0..* para 0..*) e obrigatoriamente a Template (N para 1). Seus atributos incluem codigoUnico, dataEmissao, localEmissao, status (StatusCertificadoEnum), arquivoPDF e qrCode, com os métodos gerarPDF(), assinar(assinatura) e enviarEmail(envio). O envio por e-mail é modelado pela classe EnvioEmail (destinatario, status em StatusEnvioEnum, tentativas, dataEnvio, mensagemPersonalizada), com o método enviar(), em associação 1 para 0..* com Certificado. A assinatura digital do certificado é vinculada diretamente pela relação “assinadoCom” entre Certificado e AssinaturaDigital (1 para 1).

Para auditoria, LogSistema registra eventos genéricos (tipo, descricao, dataHora, ip) vinculados ao usuário e à organização (1 para 0..* em ambas), enquanto LogAuditoria armazena ações específicas sobre certificados (acao, dataHora, local), com o método registrar(usuario), também vinculada ao usuário e à organização em 1 para 0..*, permitindo rastrear quem emitiu ou alterou cada certificado, em qual local e para qual inquilino. Sessao gerencia tokens de autenticação ativos (tokenHash, ipAddress, ativo), vinculada a Usuario em 1 para 0..*. FilaProcessamento gerencia tarefas assíncronas (tipoAcao, status em StatusFilaEnum, prioridade, tentativas para retentativas automáticas (RNF021), erro da última execução e criadoEm), e Importacao registra o histórico de importações de dados (nomeArquivo, status em StatusImportacaoEnum), ambas associadas à organização para isolamento multi-tenant.

O suporte técnico é tratado por ChamadoSuporte (titulo, descricao, dataAbertura, status em StatusChamadoEnum, dataFechamento), com os métodos abrir(usuario) e fechar(), associado em 1 para 0..* ao usuário que abriu o chamado (relação nomeada “abriu”) e em 1 para 0..* à organização. As configurações do sistema são centralizadas em ConfiguracaoSistema (envioAutomatico, regraReemissao, mensagemPadraoEmail, termosUso, sistemaOnline), com o método aplicar(), e uma instância por organização (1 para 1) para controle individualizado de cada inquilino.

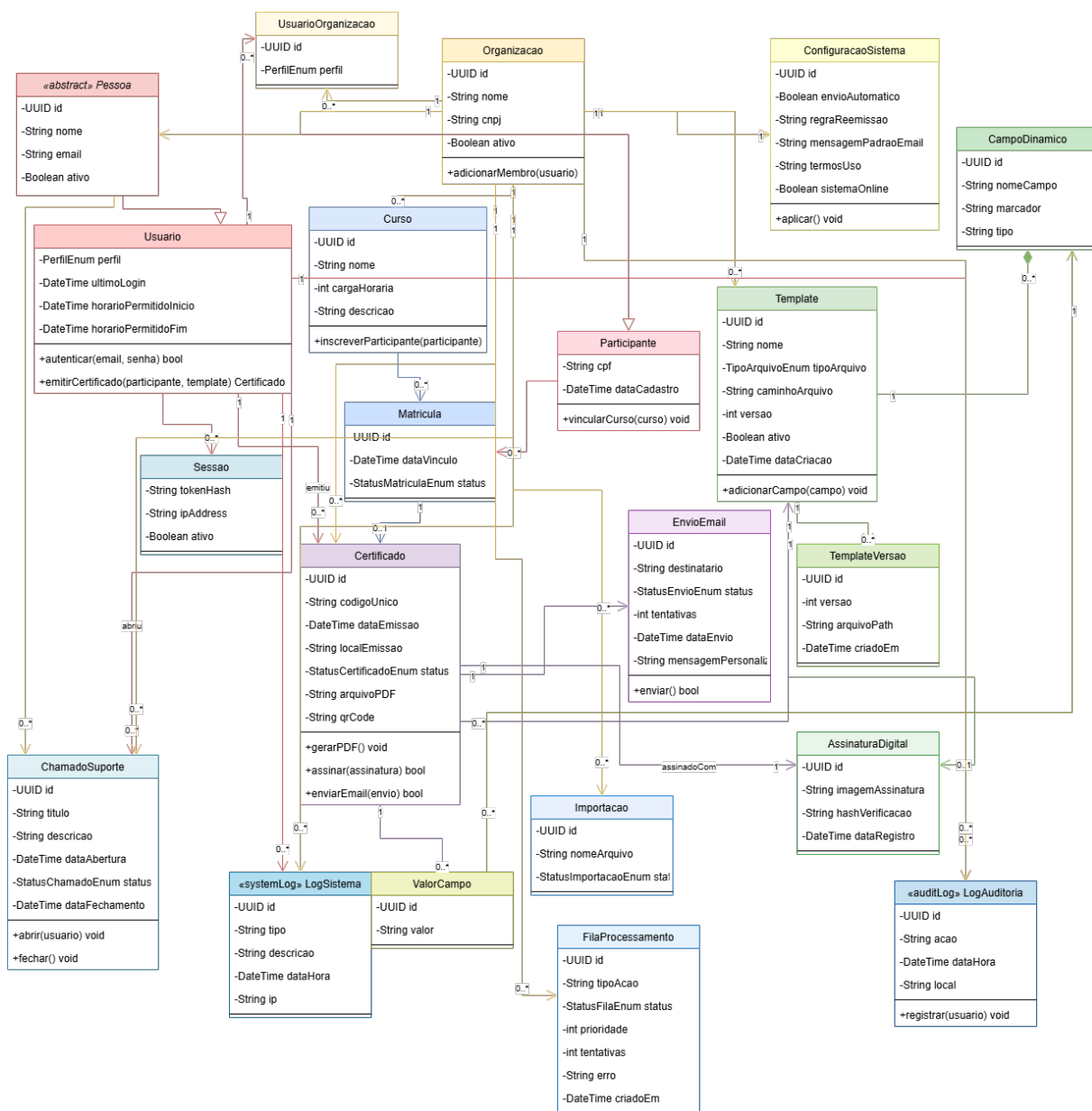


Figura 2 – Diagrama de Classes — Sistema de Certificados

3.7 Arquitetura do Sistema

A arquitetura do sistema foi projetada para atender aos requisitos de independência entre camadas, testabilidade e facilidade de manutenção. Optou-se por uma abordagem de duas aplicações independentes — frontend e backend — que se comunicam exclusivamente via API REST. Essa separação permite que cada aplicação seja desenvolvida, testada, implantada e escalada de forma independente, sem acoplamento tecnológico entre as camadas de apresentação e lógica de negócio.

O backend adota uma arquitetura em quatro camadas, organizadas pelo princípio de inversão

de dependências, em que camadas externas dependem de camadas internas:

1. **Apresentação (*App*)** — *Route Handlers* do Next.js que expõem os endpoints REST (`/api/certificates`, `/api/templates`, `/api/verify`) e o utilitário `handleApiError`, que converte exceções de domínio e erros de validação Zod em respostas HTTP padronizadas. Esta camada recebe a requisição, faz o *parse* do JSON, aciona a validação do DTO e delega a execução ao *use case*.
2. **Aplicação (*Application*)** — *Use cases* que orquestram o fluxo de negócio (Emitir, Listar, Verificar, Gerar PDF, Gerenciar Templates) e **DTOs** definidos como esquemas Zod (`emitCertificateDTO`, `updateValidityDTO`, `verifyCertificateDTO`, etc.), que validam os dados de entrada antes de alcançarem o domínio.
3. **Domínio (*Domain*)** — Núcleo da aplicação, sem dependências externas. Contém as **entidades** (`Certificate`, `Template`) com suas regras de negócio encapsuladas, **interfaces de repositório** (`ICertificateRepository`, `ITemplateRepository`), **interfaces de serviço** (`IPDFService`), **Value Objects** (`VerificationCode`, com geração e validação de código único) e **erros de domínio** (`CertificateNotFoundError`, `InvalidCertificateDataError`, etc.).
4. **Infraestrutura (*Infrastructure*)** — Implementações concretas das interfaces definidas no domínio: `SupabaseCertificateRepository`, `SupabaseTemplateRepository` e `PDFKitService`, além dos clientes de acesso ao banco PostgreSQL gerenciado pelo Supabase.

O fluxo de dados percorre as camadas da seguinte forma: uma requisição HTTP chega ao *Route Handler* (Apresentação), que valida o corpo da requisição contra o esquema Zod (Aplicação) e invoca o *use case*. O *use case* orquestra as operações de negócio utilizando entidades e *Value Objects* do Domínio, e persiste os dados por meio das interfaces de repositório — cujas implementações concretas (Infraestrutura) são injetadas via construtor. A resposta segue o caminho inverso até ser retornada ao cliente. Esse isolamento garante que as regras de negócio permaneçam independentes de frameworks, bancos de dados ou serviços externos. A inversão de dependência é obtida por meio de interfaces definidas no domínio e implementadas na infraestrutura — por exemplo, `ICertificateRepository` e `IPDFService` são contratos que o *use case* de emissão consome sem conhecer a implementação concreta. Entre as limitações desta abordagem, destaca-se o aumento do número de arquivos e interfaces, o que pode

e elevar a complexidade inicial do projeto em comparação com arquiteturas mais simples como *controllers* monolíticos.

O frontend adota uma arquitetura em camadas com React, composta por quatro camadas que separam responsabilidades de forma clara. A **View** (apresentação) contém os componentes React que renderizam a interface do usuário, divididos em páginas (`views/`) e componentes reutilizáveis (`components/`), incluindo os do `shadcn/ui`. A **ViewModel** (lógica de apresentação) é formada por custom hooks (`useCertificatesViewModel`, `useDashboardViewModel`, etc.) que encapsulam estado (`useState`), efeitos colaterais (`useEffect`) e dados derivados (`useMemo`), expondo um objeto com estado e manipuladores para a camada View. O **Service** (infraestrutura) centraliza a comunicação com o backend: o módulo `api.ts` consome a API REST via `fetch`, e o módulo `envios.ts` gerencia a fila local de envios de e-mail no `localStorage` — armazenando apenas dados operacionais (nome e e-mail do destinatário, *status* da entrega), sem tokens de autenticação ou credenciais. O **Model** (domínio) reúne as interfaces TypeScript que definem as entidades do sistema (`Certificate`, `Template`, `VerifyResult`, `EnvioEmail`). O TanStack Router orquestra a navegação entre as Views, e o Tailwind CSS gerencia a estilização em todas as camadas de apresentação. Essa separação permite testar a lógica de apresentação independentemente da interface, utilizando Jest para validar os ViewModels sem depender de um navegador. Como limitação, o aumento do número de ViewModels eleva a quantidade de arquivos e hooks, o que pode tornar a navegação entre camadas mais custosa em projetos de grande escala.

O fluxo de comunicação entre as duas aplicações ocorre da seguinte forma: o frontend SPA, executado no navegador, faz requisições HTTPS aos *Route Handlers* do backend por meio do módulo `api.ts` (camada Service), que encapsula a `fetch` API nativa. As requisições são autenticadas via token JWT gerenciado pelo Supabase Auth e trafegam sobre HTTPS (TLS 1.2), conforme RNF044. O backend processa a requisição através de suas quatro camadas — Apresentação, Aplicação, Domínio e Infraestrutura — e retorna a resposta padronizada ao frontend. Esse fluxo ponta a ponta está representado nos diagramas das Figuras 3 e 4, onde a seta "HTTPS REST + JWT" conecta o frontend SPA aos *Route Handlers* do backend.

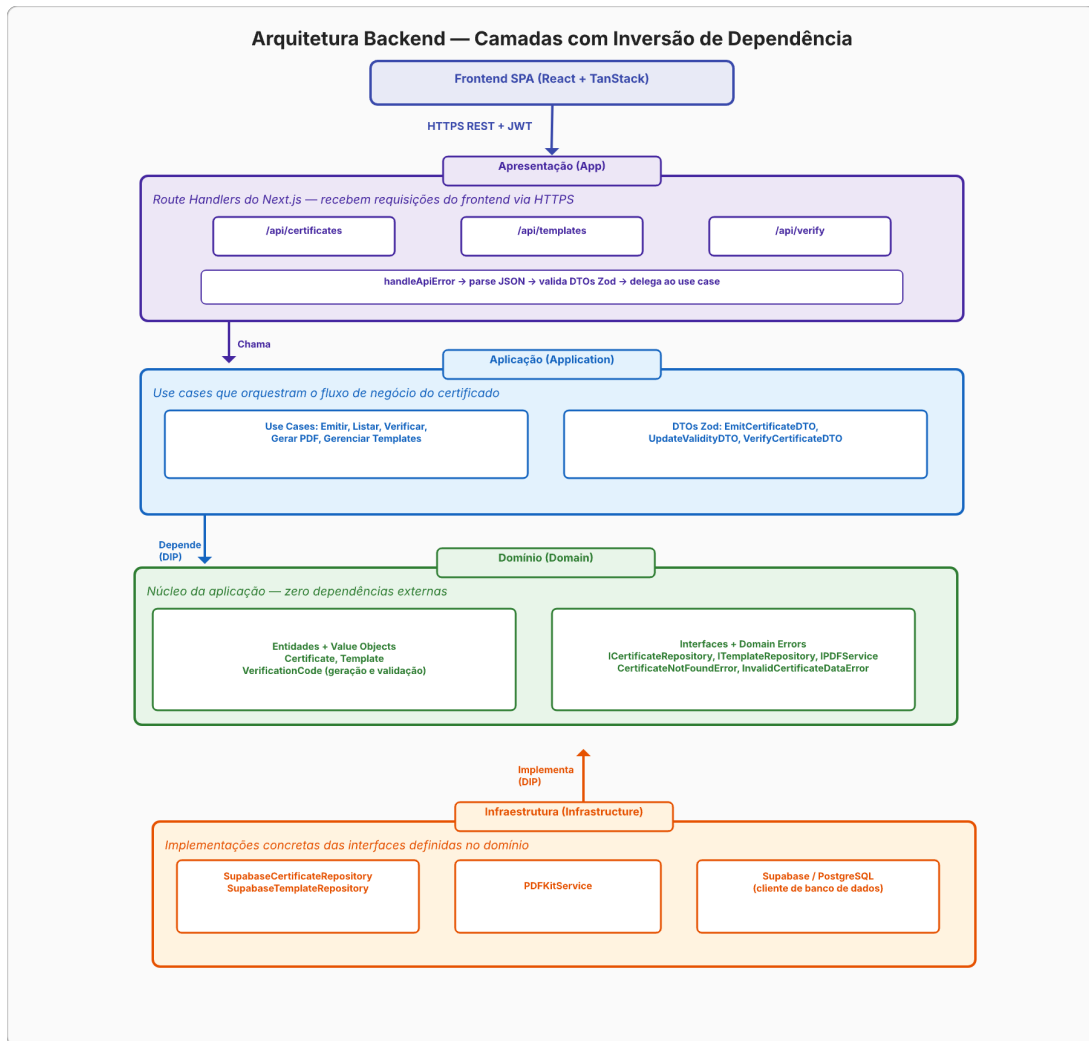


Figura 3 – Arquitetura do Backend — Quatro camadas (App, Application, Domain, Infrastruc-ture) com DTOs, Value Objects e fluxo de dependências

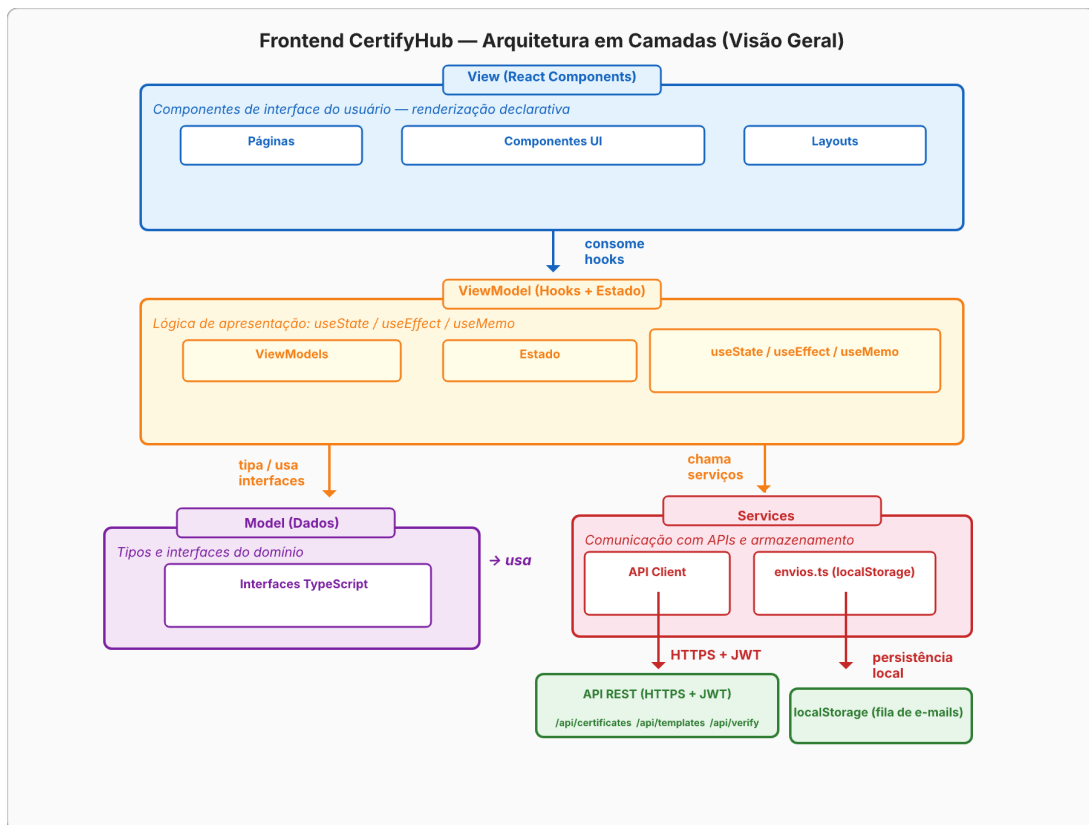


Figura 4 – Arquitetura do Frontend — Quatro camadas (View, ViewModel, Service, Model) com React

3.8 Diagrama Entidade-Relacionamento

O Diagrama Entidade-Relacionamento da Figura 5 modela a estrutura do banco de dados do sistema com 21 entidades que refletem os módulos funcionais definidos nos requisitos. A notação adotada é a pé-de-galinha (Engenharia da Informação), reconhecível pelos símbolos de barra (—) indicando o lado “1” e pelo símbolo de três pontas (—<) indicando o lado “N” (muitos) de cada relacionamento. As cardinalidades são expressas exclusivamente por esses símbolos ao longo das linhas de relacionamento. Todas as entidades utilizam UUID como identificador primário (com exceção de Sessao, que utiliza tokenHash), garantindo escalabilidade e independência entre ambientes.

A entidade central é Organizacao, que implementa o padrão multi-tenant. A estratégia adotada é o isolamento por coluna organizacional (*column-based isolation*): todas as entidades que armazenam dados pertencentes a um inquilino específico possuem uma chave estrangeira `organizacao_id` para a entidade Organizacao. Essa abordagem é aplicada diretamente em Cursos, Templates, Certificados, Pessoas, Importacoes, FilaProcessamento, LogAuditoria,

LogSistema, ChamadosSuporte, ConfiguracoesSistema (1:1), e indiretamente por meio da cadeia de chaves estrangeiras em entidades como TemplateVersao e CampoDinamico (via Templates), ValorCampo e EnvioEmail (via Certificados), AssinaturaDigital (via Certificados, 1:1, e via Templates, 1:0..1) e Matriculas (via Cursos). Dessa forma, consultas realizadas por uma organização retornam exclusivamente os dados a ela vinculados, sem risco de vazamento entre inquilinos.

O controle de acesso é modelado pela tabela associativa UsuarioOrganizacao, que resolve a relação muitos-para-muitos entre Usuarios e Organizacao. Um usuário pode pertencer a múltiplas organizações com perfis distintos (administrador, emissor, visualizador), e cada organização pode ter vários usuários vinculados. As cardinalidades 1:N de Usuarios e Organizacao para UsuarioOrganizacao garantem flexibilidade no gerenciamento de permissões sem acoplar a autenticação ao inquilino.

A hierarquia de herança tem Pessoa como entidade abstrata que define os atributos comuns (nome, email, ativo), generalizada em Usuario (credencial de acesso com perfil, último login e horários permitidos inicio/fim) e Participante (dados específicos como CPF e data de cadastro). Sessao gerencia tokens de autenticação ativos (tokenHash como chave primária, ipAddress e ativo), vinculada a Usuarios em 1:N.

O módulo de certificados tem Certificados como entidade principal, associada a Organizacao e Templates em 1:N, e a Matriculas em 1:1. ValorCampo armazena os pares nome/valor preenchidos durante a emissão (em 1:N com certificados) e referenciam CampoDinamico, que define os campos editáveis do template com tipo e marcador. EnvioEmail registra o histórico de tentativas de entrega por e-mail. AssinaturaDigital armazena a referência de assinatura digital vinculada a cada certificado (relação 1:1), registrando a imagem da assinatura, o hash de verificação e a data de registro. Cada template pode opcionalmente definir uma assinatura padrão (relação 1:0..1), que servirá como referência visual do modelo. A relação muitos-para-muitos entre Participantes e Cursos é resolvida pela tabela associativa Matriculas, que armazena o vínculo entre participante e curso, permitindo rastrear historicamente cada matrícula. Cada Matricula possui status (ativo, concluído, cancelado, trancado) e data de vínculo.

Os templates seguem um versionamento modelado por Templates (1:N com TemplateVersao, que registra a data de criação de cada versão), onde cada versão armazena o arquivo do layout e o histórico de alterações. O template também possui data de criação para rastrear seu cadastro. CampoDinamico define os campos editáveis de cada template (nome do campo,

marcador e tipo) em uma relação de composição, permitindo que templates personalizem os dados coletados na emissão. Certificados referenciam o template utilizado na emissão (relação N:1), garantindo que certificados emitidos anteriormente mantenham o vínculo com o template independentemente de versões futuras.

Para auditoria e rastreabilidade, LogAuditoria registra ações sobre certificados (emissão, reemissão, cancelamento) com data/hora e local, vinculadas à organização e ao usuário responsável, com cardinalidade 1:N em ambas as relações. LogSistema complementa com um registro genérico de eventos do sistema (tipo, descrição, data/hora, IP), também isolado por organização. FilaProcessamento gerencia tarefas assíncronas (geração de PDF em lote, envio de e-mails) com tipo de ação, status, prioridade, contador de tentativas (suporte a retentativas automáticas — RNF021), registro de erro da última execução e data de criação, associada à organização em 1:N, enquanto Importacoes registra o histórico de importações de dados (participantes, matrículas) com nome do arquivo e status, realizadas por usuários de cada organização. ChamadosSuporte registra solicitações de suporte técnico (título, descrição, status), vinculadas à organização e ao usuário solicitante, com cardinalidade 1:N em ambas as relações.

Em termos de restrições de integridade e otimização, as seguintes *constraints* e índices são previstos no modelo físico, alinhados aos requisitos não funcionais de desempenho e segurança da Seção 3.1:

- **Chaves únicas (UNIQUE):** `certificados.codigo_unico` — suporte à validação pública via código único (RF033–RF036); `participante.cpf` — identificação única do participante (RF054); `usuario.email` — credencial de autenticação (RF001); `organizacao.cnpj` — identificação fiscal da empresa (RF059).
- **NOT NULL obrigatórios:**
 - `pessoa.nome` e `pessoa.email`;
 - `certificado.codigo_unico`, `data_emissao` e `status`;
 - `organizacao.nome`;
 - `curso.nome` e `carga_horaria`;
 - `template.nome`;
 - `fila_processamento.tentativas`.

- **Índices recomendados:** índice único em `certificados.codigo_unico` — consulta de validação pública com requisito de resposta em até 2 segundos (RNF013); índice em `certificados.data_emissao` — filtros do dashboard (RF039–RF042); índice em `logs_auditoria.data_hora` — consultas de auditoria (RF043–RF045); índice em `matriculas.participante_id` — histórico de certificados por participante (RF057); índice em `fila_processamento.status` — consulta de tarefas pendentes pela fila de processamento (RNF016–RNF017); índice composto em `certificados.organizacao_id+data_emissao` — relatórios mensais isolados por inquilino.

Essas definições serão aplicadas na camada de persistência do sistema, garantindo que as operações de consulta crítica atendam aos SLAs de desempenho estabelecidos.

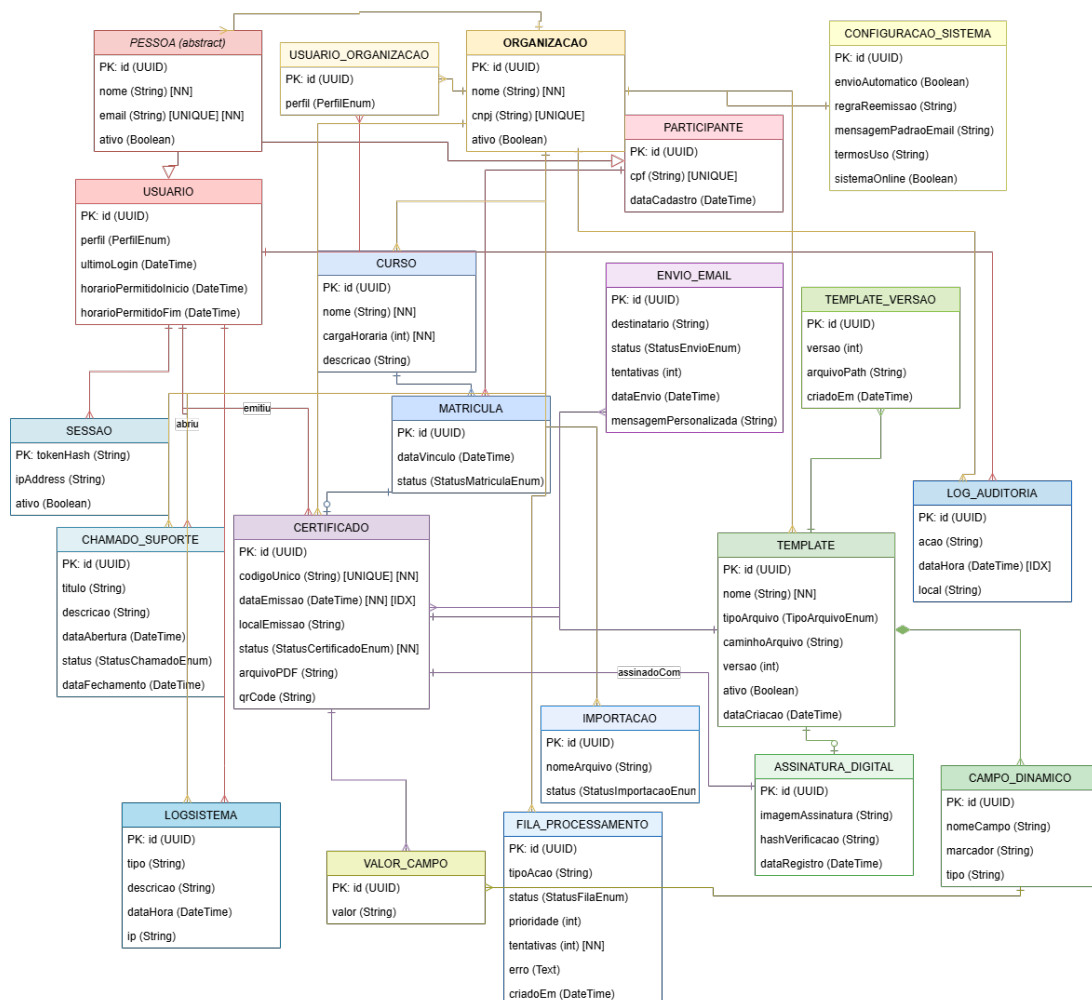


Figura 5 – Diagrama Entidade-Relacionamento (DER) do Banco de Dados

3.9 Interfaces

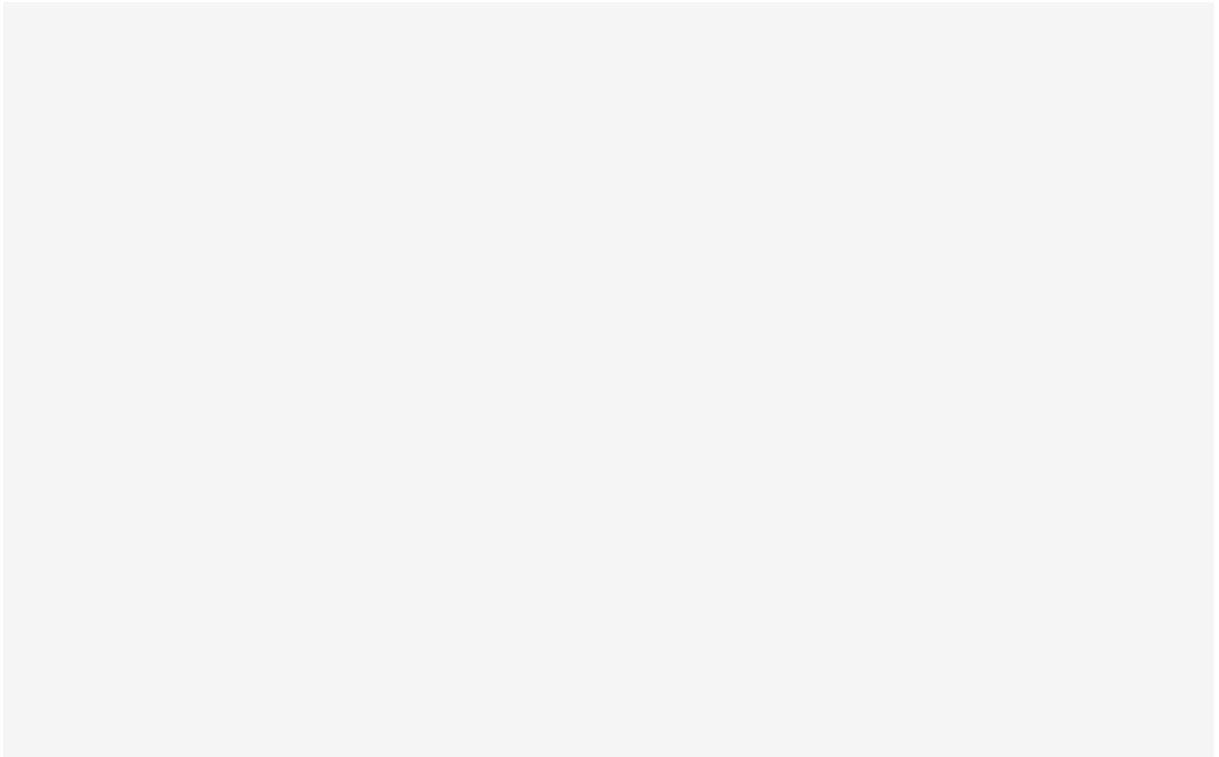


Figura 6 – Interface do Sistema — Dashboard de Certificados

4.0 Software

4.1 Implantação

O sistema é composto por duas aplicações independentes implantadas no Render: o frontend como Static Site e o backend como Web Service. Ambos utilizam auto-deploy do Render, que escuta pushes na branch main do repositório GitHub, compila e publica automaticamente a nova versão.

O frontend é uma SPA construída com Vite. Durante o deploy, o Vite gera os artefatos estáticos (HTML, CSS, JS) na pasta dist, que o Render serve diretamente. O roteamento no lado do cliente é gerenciado pelo TanStack Router via History API, sem necessidade de configuração adicional no servidor.

O backend é um servidor Node.js (Next.js) executado como Web Service no Render. O Render inicia a aplicação com `npm start` e expõe a porta definida pela variável de ambiente `PORT`. O banco de dados PostgreSQL é hospedado no Supabase, que fornece um serviço gerenciado com backups automáticos e SSL obrigatório. O backend se conecta ao Supabase via

DATABASE_URL configurada no painel do Render.

A Figura 7 ilustra o fluxo de implantação: o desenvolvedor realiza push no GitHub, o Render detecta a mudança na branch main, executa o build de cada serviço e publica a nova versão automaticamente.

Diagrama de Implantação — Render

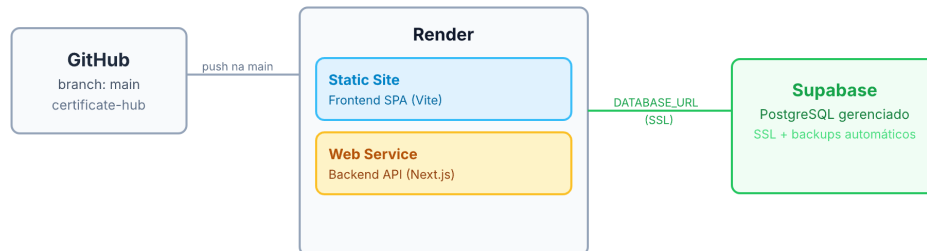


Figura 7 – Diagrama de Implantação — Fluxo de deploy no Render a partir do GitHub, com frontend como Static Site, backend como Web Service e banco PostgreSQL gerenciado pelo Supabase

4.2 Testes

Conforme definido no objetivo específico da Seção 1.2.2, foram planejados e executados testes unitários para validação dos módulos implementados no escopo do MVP. Os casos de teste foram elaborados com base nos critérios de aceitação apresentados na Tabela 2 e abrangem os requisitos funcionais classificados como prioritários — *Must have* [M] e *Should have* [S] — essenciais para a primeira versão do sistema.

Os testes foram classificados como unitários, uma vez que cada módulo (frontend e backend) foi testado de forma isolada, com dependências externas (banco de dados, APIs de terceiros, bibliotecas de geração de PDF) substituídas por *mocks*. Testes de integração, desempenho e segurança não foram incluídos no escopo do MVP, podendo ser abordados em versões futuras do sistema. Testes *end-to-end* (E2E) foram implementados com Playwright para validar o fluxo completo do sistema, conforme descrito na Seção 4.2.4. Todos os testes foram executados em ambiente local, com Node.js 22, e integraram o pipeline de CI/CD do Render, que executa a suíte completa a cada *push* na branch `main`.

4.2.1 Plano de Testes

Estratégia de Testes O plano de testes foi elaborado com base nos requisitos funcionais (Seção 3.1.1) e nos critérios de aceitação (Tabela 2), contemplando exclusivamente os módulos implementados no escopo do MVP. A estratégia adotada prioriza testes unitários como primeira camada de defesa, combinada com testes de integração para validação das camadas de infraestrutura e testes *end-to-end* (E2E) para verificação do fluxo completo do sistema.

- **Ferramentas:** Jest como *test runner* e *framework* de asserção; React Testing Library para testes de *ViewModels* no frontend; Playwright para testes *end-to-end*; *Mocking* manual via `jest.mock` para isolar dependências externas (Supabase, PDFKit, `fetch`).
- **Execução em CI/CD:** A suíte completa é executada automaticamente pelo Render a cada *push* na branch `main`. O pipeline valida que todos os testes passem antes da implantação.
- **Meta de cobertura:** O projeto define cobertura mínima de 75% em *Statements*, *Functions* e *Lines*, configurada no Jest e validada no pipeline de CI. As camadas de domínio e casos de uso no backend têm meta informal de 100%.
- **Massa de dados:** Os testes de repositório utilizam o cliente Supabase *mockado*, dispensando banco de dados real. Para testes das entidades de domínio, os dados são construídos em memória (*in-memory*), garantindo isolamento total entre cenários.

Casos de Teste Cada caso de teste é identificado por um código único (CT nnn) e descreve o cenário avaliado juntamente com o resultado esperado. A Tabela 17 apresenta o plano completo, organizado por módulo e tipo de teste.

Tabela 17 – Plano de testes do sistema

Caso	Descrição	Resultados Esperados
Módulo de Templates de Certificados (RF008–RF014)		
CT001	Criação de template	Template é criado com nome, descrição e <i>layout</i> mesclado aos valores padrão.
CT002	Listagem de templates	<i>Endpoint</i> retorna lista de templates cadastrados.
CT003	Obtenção de template por ID	<i>Endpoint</i> retorna template específico; lança erro <code>TemplateNotFoundError</code> se não existir.

Caso	Descrição	Resultados Esperados
CT004	Edição de template	Nome, descrição e configurações de <i>layout</i> são alterados e persistem.
CT005	Exclusão de template	Template é removido após confirmação do usuário; <i>endpoint delete</i> é chamado.
CT006	Personalização de <i>layout</i>	Configurações parciais são mescladas com os valores padrão (DEFAULT_LAYOUT).
Módulo de Geração de Certificados (RF023–RF029)		
CT007	Emissão manual de certificado	Selecionar template, preencher dados e confirmar gera certificado com código único.
CT008	Geração de PDF	PDF gerado preserva layout, cores e fontes do template; resultado é um <i>Buffer</i> binário.
CT009	Geração de PDF com cores personalizadas	PDF é gerado com <code>backgroundColor</code> , <code>primaryColor</code> e <code>borderColor</code> personalizados.
CT010	Geração de PDF sem borda	PDF é gerado corretamente com <code>showBorder = false</code> .
CT011	Erro na geração de PDF	Falha no PDFKit rejeita a <i>Promise</i> com mensagem de erro.
CT012	Obtenção de certificado por ID	<i>Endpoint</i> retorna certificado específico; lança erro <code>CertificateNotFoundError</code> se não existir.
CT013	Listagem de certificados	<i>Endpoint</i> retorna lista de certificados cadastrados.
CT014	Atualização de certificado	Nome, CPF, carga horária, curso e <code>templateId</code> são alterados individual ou simultaneamente.
CT015	Atualização de validade	Data de validade é atualizada; data passada é rejeitada.
CT016	Validação de dados de entrada	Nome vazio, CPF inválido, carga horária negativa e <code>templateId</code> não-UUID são rejeitados.
CT017	Validação de CPF	CPF com dígitos verificadores incorretos ou todos iguais é rejeitado.
CT018	Formatação de CPF	CPF é automaticamente formatado (XXX.XXX.XXX-XX) durante a digitação.
CT019	Emissão com template inexistente	<i>Endpoint</i> lança <code>TemplateNotFoundError</code> se o <code>templateId</code> não existir.
Módulo de Validação Pública e Antifraude (RF033–RF036)		
CT020	Validação via código único	Código válido retorna certificado com status autêntico e válido.
CT021	Certificado expirado	Certificado com validade vencida é retornado como autêntico, porém inválido.

Caso	Descrição	Resultados Esperados
CT022	Código não encontrado	Código inexistente é rejeitado e a validação retorna certificado não localizado.
CT023	<i>Case insensitive</i> na consulta	Código digitado em minúsculo é convertido para maiúsculo antes da consulta no banco.
CT024	Remoção de máscara do CPF	CPF com pontuação tem os caracteres não numéricos removidos antes da consulta.
CT025	Tratamento de erro na validação	Erros retornados pela API são exibidos ao usuário e o estado de carregamento é gerenciado corretamente.
Módulo de <i>Dashboard</i> e Gestão (RF039–RF041)		
CT026	Listagem de certificados	Tabela exibe certificados carregados da API com gerenciamento do estado de carregamento.
CT027	Filtro por nome, CPF e código	Digitar texto no campo de busca filtra a lista em tempo real.
CT028	Cópia do código de verificação	Botão copia o código para a área de transferência.
CT029	Atualização de validade na interface	Usuário altera data de validade; a requisição é enviada ao servidor e a lista é recarregada.
CT030	Estatísticas do <i>dashboard</i>	Painel exibe total de certificados, ativos, templates e envios.
CT031	Cálculo de certificados ativos	Certificados expirados são excluídos da contagem de ativos.
Entidades de Domínio		
CT032	Criação de certificado válido	Entidade <code>Certificate</code> é criada com todos os campos obrigatórios.
CT033	Validações da entidade <code>Certificate</code>	Nome vazio, CPF inválido, curso vazio, carga horária zero/negativa e <code>templateId</code> vazio disparam erro de domínio.
CT034	Data de validade futura	Certificado com validade futura retorna <code>isValid = true</code> .
CT035	Data de validade passada	Certificado com validade passada retorna <code>isValid = false</code> .
CT036	Criação de template válido	Entidade <code>Template</code> é criada com <i>layout</i> mesclado aos valores <i>default</i> .
CT037	Validações da entidade <code>Template</code>	Nome vazio ou com apenas espaços dispara erro de domínio.
CT038	Geração de código de verificação	Código gerado possui exatamente 8 caracteres do <i>charset</i> definido.
CT039	Comparação de códigos	Método <code>equals()</code> compara dois códigos corretamente; <i>case insensitive</i> .

Caso	Descrição	Resultados Esperados
CT040	Erros de domínio	Classes de erro (<code>CertificateNotFoundError</code> , <code>InvalidCPFError</code> , etc.) possuem mensagens e <i>status</i> HTTP adequados.
CT041	Restauração a partir de persistência	Dados vindos do banco são convertidos corretamente para instâncias das entidades.
Camada de Infraestrutura		
CT042	Repositório de certificados — consultas	Operações <code>findById</code> , <code>findByCPFAndCode</code> e <code>findAll</code> retornam certificados ou <code>null</code> .
CT043	Repositório de certificados — escrita	Operações <code>create</code> , <code>update</code> e <code>delete</code> persistem no Supabase e tratam erros.
CT044	Repositório de templates — consultas	Operações <code>findById</code> e <code>findAll</code> retornam templates ou <code>null</code> com <i>fallback</i> de valores.
CT045	Repositório de templates — escrita	Operações <code>create</code> , <code>update</code> e <code>delete</code> persistem no Supabase e tratam erros.
CT046	Geração de PDF com PDFKit	PDF é gerado como <i>Buffer</i> ; cores e bordas são aplicadas; erro no PDFKit é tratado.
CT047	Tratamento de erros da API	Erros de domínio (404, 400), erros Zod (400) e erros inesperados (500) são mapeados para <i>status code</i> HTTP corretos.
Utilitários (Frontend)		
CT048	Chamadas HTTP da API	Métodos <i>GET</i> , <i>POST</i> , <i>PUT</i> , <i>DELETE</i> encapsulam <code>fetch</code> com tratamento de erro e <i>parsing</i> JSON.
CT049	Serviço de envios (<i>localStorage</i>)	Operações de listagem e adição de registros de envio persistem no <i>localStorage</i> com tratamento de erro.
CT050	Deteção de <i>mobile</i>	Hook <code>useIsMobile</code> retorna <code>true</code> para larguras menores que 768 px; <i>listener</i> é limpo no desmonte.
CT051	Função <code>isExpired</code>	Datas passadas retornam <code>true</code> ; datas futuras e presentes retornam <code>false</code> .
Testes Não-Funcionais		
CT052	Carga — Geração simultânea de PDFs	Múltiplas requisições concorrentes de geração de PDF não causam falhas em cascata; o sistema permanece responsivo.
CT053	Segurança — Acesso não autenticado	Endpoints de criação, edição e exclusão de templates e certificados rejeitam requisições sem token de autenticação (status 401).

Caso	Descrição	Resultados Esperados
CT054	Segurança — Força bruta na validação	Requisições repetidas ao endpoint de validação são limitadas (<i>rate limiting</i>); o sistema bloqueia temporariamente após múltiplas tentativas inválidas.
CT055	Acessibilidade — Navegação por teclado	Todos os campos do formulário de validação pública são acessíveis via tecla Tab; o botão de busca pode ser acionado via Enter.
Testes End-to-End (E2E)		
CT056	Fluxo completo de emissão e validação	Usuário cria template, emite certificado e valida o código no portal público; todas as etapas são concluídas com sucesso. (Automatizado com Playwright.)
CT057	Validação via <i>QR Code</i>	<i>QR Code</i> gerado no PDF, quando escaneado, redireciona para a URL de validação com o código correto como parâmetro.
Cenários de Resiliência		
CT058	Indisponibilidade do Supabase	Cliente Supabase retorna erro de conexão; o sistema exibe mensagem amigável ao usuário sem interromper as demais funcionalidades.
CT059	Concorrência — Edição simultânea	Dois administradores editam o mesmo template ao mesmo tempo; o último salvamento prevalece e o conflito é registrado em log.
CT060	<i>Timeout</i> na geração de PDF	Geração de PDF excede o tempo limite; o sistema cancela a operação e notifica o usuário sobre a falha.

A Tabela 17 relaciona 60 casos de teste planejados, incluindo cenários funcionais, não-funcionais, *end-to-end* e de resiliência. Cada caso pode ser validado por um ou mais testes automatizados, totalizando 276 testes unitários implementados com o framework Jest. As subseções seguintes detalham os resultados da execução em cada módulo.

4.2.2 Frontend (Certificate Hub)

O frontend contém 11 suítes de testes com 108 testes unitários, abrangendo as *ViewModels* dos módulos de templates (RF008–RF014), certificados (RF023–RF029), *dashboard* (RF039–RF041) e validação pública (RF033–RF036), além dos serviços de API, componentes de formulário e dos modelos de domínio.

A métrica de cobertura, gerada automaticamente pelo Jest por meio da ferramenta Istanbul, indica a proporção do código-fonte que é exercitada durante a execução dos testes. O relatório é organizado em quatro categorias: *Statements* (comandos executados), *Branches* (ramos de decisão, como `if/else`), *Functions* (funções chamadas) e *Lines* (linhas de código percorridas). A Figura 8 apresenta o relatório de cobertura gerado para o frontend.

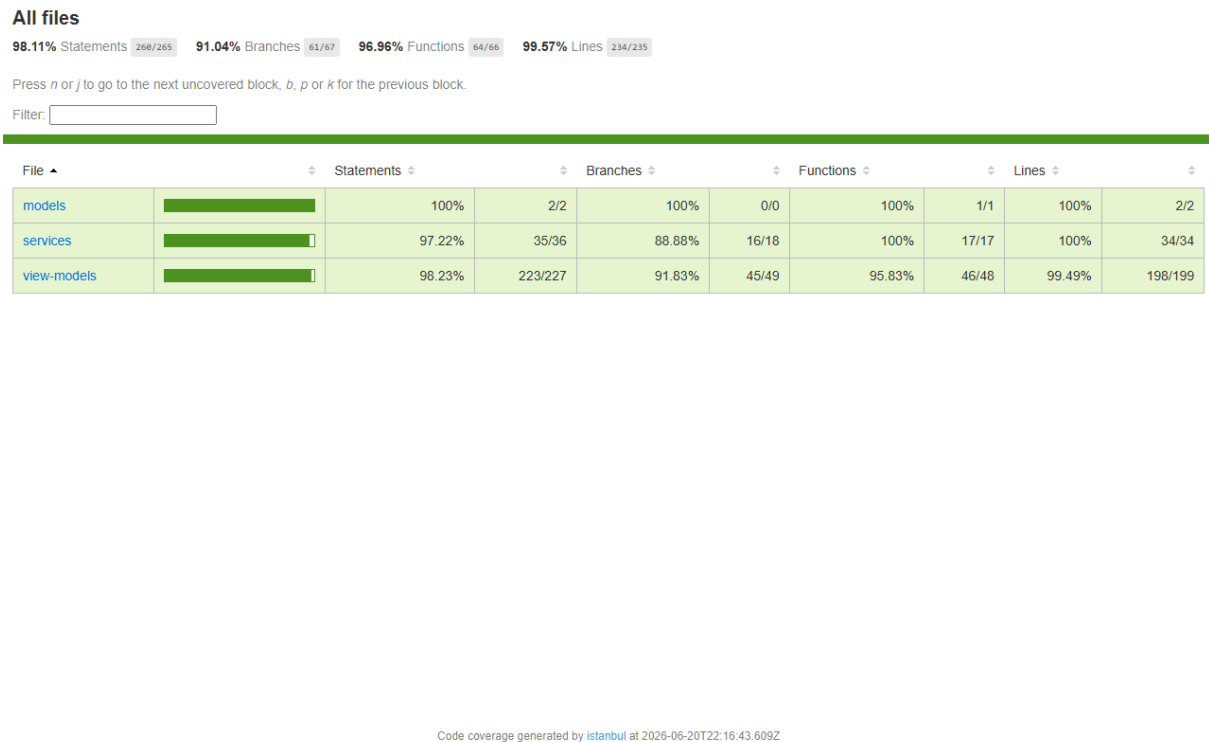


Figura 8 – Relatório de cobertura dos testes — frontend

Conforme ilustrado, o frontend atingiu 98,11% de cobertura de comandos (*Statements*), 91,04% de ramos (*Branches*), 96,96% de funções (*Functions*) e 99,57% de linhas (*Lines*). O diretório `models` obteve cobertura total (100%), enquanto `services` alcançou 97,22% e `view-models` 98,23% em comandos. A cobertura de ramos em `services` (88,88%) reflete a presença de tratamentos condicionais de erro que não foram todos exercitados nos testes.

4.2.3 Backend (Certificate Server)

O backend contém 19 suítes de testes com 168 testes unitários, cobrindo casos de uso (emissão e reemissão RF023–RF028, validação RF033–RF036, templates RF008–RF014), entidades de domínio (`Certificate`, `Template`, `VerificationCode`), repositórios (`Supabase`), serviços (`PDFKit`) e *middleware* de autenticação e *rate limiting* — principais camadas da arquitetura limpa adotada no MVP.

A Figura 9 apresenta o relatório de cobertura gerado pelo Jest para o backend.

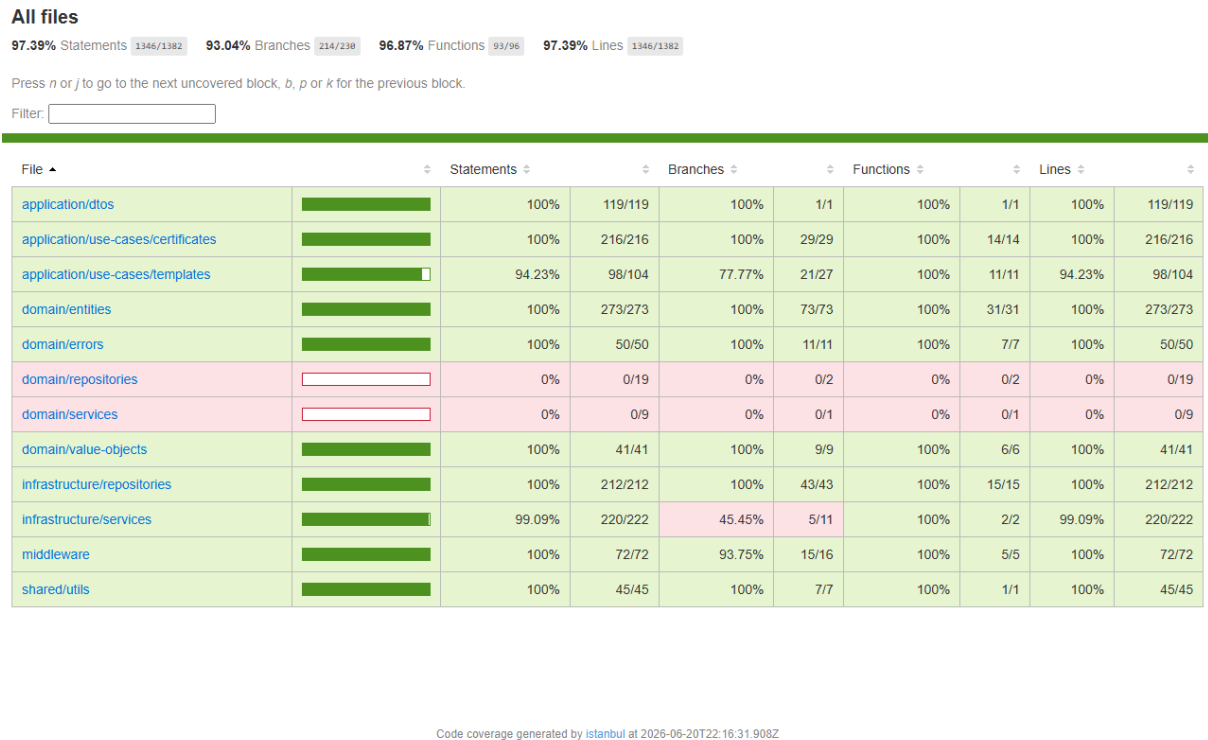


Figura 9 – Relatório de cobertura dos testes — backend

O backend atingiu 97,62% de cobertura de comandos, 95,38% de ramos, 96,66% de funções e 97,62% de linhas. As camadas de *application/dtos*, *application/use-cases*, *domain/entities* e *infrastructure/repositories* apresentaram cobertura integral (100%), evidenciando que as regras de negócio e a lógica de persistência foram amplamente exercitadas pelos testes. A camada *infrastructure/services* registrou 99,05% em comandos e 45,45% em ramos, diferença atribuída às condicionais de configuração do PDFKit que não foram totalmente cobertas. As interfaces *domain/repositories* e *domain/services* aparecem com 0% por se tratarem de contratos abstratos (interfaces) sem implementação concreta a ser testada diretamente.

4.2.4 Testes End-to-End

Para validar a integração entre frontend e backend em cenário realista, foi implementado um teste *end-to-end* com Playwright que percorre o fluxo completo: criar um template, emitir um certificado e verificar a autenticidade pela página pública. O teste é executado contra o servidor de desenvolvimento local (Vite na porta 5173) e depende do backend rodando simultaneamente (Next.js na porta 3000). Todo o processo é orquestrado automaticamente pelo Playwright, que inicia o frontend via `npm run dev` antes da execução.

O teste demonstra que o sistema suporta o fluxo crítico de ponta a ponta sem intervenção manual. Durante o desenvolvimento, foi identificado e corrigido um erro de configuração de CORS no backend (`next.config.ts`), que impedia a comunicação entre os serviços em `localhost`. A Figura 10 apresenta o relatório gerado pelo Playwright, que exibe o passo a passo da execução, incluindo as capturas de tela de cada etapa e o resultado final (aprovado).

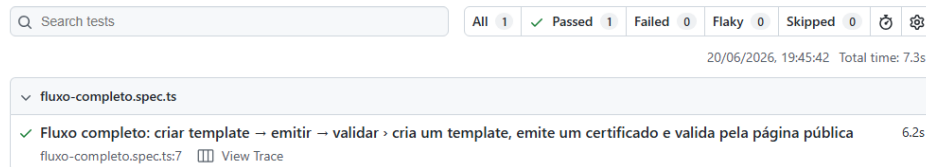


Figura 10 – Relatório do teste end-to-end gerado pelo Playwright

4.2.5 Limitações

Embora a cobertura geral tenha atingido percentuais elevados, algumas limitações devem ser registradas. O módulo de autenticação e controle de acesso (RF001–RF007) possui testes apenas para o *middleware* de autenticação, mas o fluxo completo de login, redefinição de senha e gerenciamento de sessões ainda não foi testado. Testes de integração com o banco de dados Supabase não foram realizados — os repositórios foram testados com o cliente Supabase *mockado*. Testes de acessibilidade (*allly*) foram implementados apenas para o formulário de validação. O teste E2E atualmente cobre apenas o fluxo principal (*happy path*); cenários de erro, como template ausente ou CPF inválido, não são abordados. Cenários de carga e desempenho foram testados para geração de PDF, porém não para os demais *endpoints*. Essas lacunas representam oportunidades de melhoria para versões futuras do sistema.

4.2.6 Resumo Geral

A Tabela 18 consolida os resultados dos testes em ambos os módulos, incluindo as métricas de cobertura. Ambos os módulos superaram o limiar mínimo de 75% de cobertura definido na configuração do Jest e validado no pipeline de CI.

Tabela 18 – Resumo geral dos testes

Módulo	Suítes	Testes	Cobertura	Tempo
Frontend (Certificate Hub)	11	108	98,11%	18,92 s
Backend (Certificate Server)	19	168	97,62%	7,64 s
Total	30	276	–	26,56 s

5.0 Considerações Finais

Síntese, limitações e sugestões.

Referências

- [1] RAHMAN, K. A.; AHMED, S. A. A systematic literature review on qr code–based systems for academic credential verification. *e-Jurnal Penyelidikan dan Inovasi*, v. 13, n. 1, p. 300–315, 2026.
- [2] LORIEN, A.; WELLEM, T. Implementasi sistem otentikasi dokumen berbasis quick response (qr) code dan digital signature. *Jurnal RESTI (Rekayasa Sistem dan Teknologi Informatika)*, v. 5, n. 4, p. 663–671, 2021.
- [3] KULKARNI, R. et al. A secure certificate authentication system using cryptographic hashing and qr code verification. *International Journal of Sciences and Innovation Engineering*, v. 2, n. 10, p. 1407–1414, 2025. Disponível em: <<https://ijsci.com/index.php/home/article/view/840>>.
- [4] OLAMIDE, O. O. et al. Design and implementation of a certificate verification system using quick response (qr) code. *LAUTECH Journal of Computing and Informatics*, v. 2, n. 1, p. 35–40, 2021. Disponível em: <<http://laujci.lautech.edu.ng/index.php/laujci/article/view/36>>.
- [5] KORA, H. B.; MANITA, M. Modern front-end web architecture using react.js and next.js. *University of Zawia Journal of Engineering Sciences and Technology*, v. 2, n. 1, p. 1–13, 2024.
- [6] MICROSOFT. *TypeScript Documentation — TypeScript 5.x*. 2025. Disponível em: <<https://www.typescriptlang.org/docs/>>.
- [7] LABS, T. *Tailwind CSS Documentation*. 2025. Disponível em: <<https://tailwindcss.com/docs>>.
- [8] SHADCN. *shadcn/ui Documentation*. 2025. Disponível em: <<https://ui.shadcn.com/docs>>.
- [9] LINSLEY, T.; CONTRIBUTORS, T. *TanStack Router Documentation*. 2025. Disponível em: <<https://tanstack.com/router/latest>>.
- [10] BLUEBILL1049; CONTRIBUTORS, R. H. F. *React Hook Form Documentation*. 2025. Disponível em: <<https://react-hook-form.com/>>.

- [11] KOWALSKI, E. *Sonner — Toast Component for React*. 2025. Disponível em: <<https://sonner.emilkowal.ski/>>.
- [12] GROUP, R. *Recharts Documentation*. 2025. Disponível em: <<https://recharts.org/en-US/>>.
- [13] CONTRIBUTORS *date-fns. date-fns Documentation*. 2025. Disponível em: <<https://date-fns.org/docs/>>.
- [14] TEAM, V. *Vite Documentation*. 2025. Disponível em: <<https://vitejs.dev/docs/>>.
- [15] HACKS, C. *Zod Documentation*. 2025. Disponível em: <<https://zod.dev/>>.
- [16] FOUNDATION, O. *Node.js Documentation*. 2025. Disponível em: <<https://nodejs.org/docs/latest/api/>>.
- [17] VERCEL. *Next.js Documentation*. 2025. Disponível em: <<https://nextjs.org/docs>>.
- [18] SUPABASE. *Supabase Documentation*. 2025. Disponível em: <<https://supabase.com/docs>>.
- [19] GROUP, P. G. D. *PostgreSQL Documentation*. 2025. Disponível em: <<https://www.postgresql.org/docs/16/>>.
- [20] FOLIOJS. *PDFKit — A JavaScript PDF Generation Library for Node and the Browser*. 2025. Disponível em: <<https://pdfkit.org/>>.
- [21] FOUNDATION, O. *uuid — npm package*. 2025. Disponível em: <<https://www.npmjs.com/package/uuid>>.
- [22] RENDER. *Render Documentation*. 2025. Disponível em: <<https://render.com/docs>>.
- [23] FOUNDATION, O. *Jest Documentation*. 2025. Disponível em: <<https://jestjs.io/docs/>>.
- [24] FOUNDATION, O. *ESLint Documentation*. 2025. Disponível em: <<https://eslint.org/docs/latest/>>.
- [25] CONTRIBUTORS, P. *Prettier Documentation*. 2025. Disponível em: <<https://prettier.io/docs/en/>>.

- [26] CONSERVANCY, S. F. *Git Documentation*. 2025. Disponível em: <<https://git-scm.com/doc>>.
- [27] GITHUB, I. *GitHub Documentation*. 2025. Disponível em: <<https://docs.github.com>>.
- [28] MICROSOFT. *Visual Studio Code Documentation*. 2025. Disponível em: <<https://code.visualstudio.com/docs>>.
- [29] KRÄMER, F. *Simple but useful: Use Case Tables*. 2024. Disponível em: <<https://florian-kraemer.net/software-architecture/2024/02/28/Use-Case-Tables.html>>.

6.0 Anexos

6.1 Declaração de Entrega do Código-Fonte

Declaro que o código-fonte foi entregue [...]

Assinatura: _____

6.2 Declaração de Submissão ao NIT

Declaro que o software foi submetido ao NIT [...]

Assinatura: _____